XIMOD Architect — XML Builder

Benutzerhandbuch

Das XML-Installationsprogramm, entwickelt für Ihre Mods.

Skyrim · Fallout · Starfield · Oblivion · Morrowind

Version 2.1.0

Plattformübergreifend: Windows · Linux · macOS



Inhalt

1. Überblick 5

1.1 Was ist XIMOD Architect? 5

1.2 Das FOMOD-Format 5

1.3 Installation und Anforderungen 6

Windows 6

Linux 6

macOS 6

1.4 Dateistruktur 6

2. Erste Schritte 8

2.1 Starten der Anwendung 8

2.2 Übersicht über die Benutzeroberfläche 8

2.3 Ein neues Projekt erstellen 9

2.4 Projekt öffnen und speichern 9

2.5 Erster Start 9

2.6 Freie Fenster und Tastaturnavigation 10

3. Die Menüs 11

3.1 Menü „Datei“ 11

3.2 Menü „Bearbeiten“ 12

3.3 Menü „Projekt“ 12

3.4 Menü „Werkzeuge“ 13

3.5 Menü „Hilfe“ 14

4. Funktionen im Detail 15

4.1 Die Registerkarte „Mod-Info“ 15

4.2 Die Registerkarte „Installationsschritte“ 15

Gruppen und Auswahltypen 16

Optionen und Standardtypen 16

Plugin-Dateien 17

Gruppen-/Seitenziel 17

Bedingungsflags 17

4.3 Die Registerkarte „Erforderliche Installationen“ 18

4.4 Die Registerkarte „Bedingte Installationen“ 18

4.5 Das Fenster „Einstellungen“ 19

4.6 Skripte vor und nach dem Speichern 20

Beispiel — Skript vor dem Speichern (Windows) 20

Beispiel — Skript nach dem Speichern (Windows) 20

Entsprechung unter Linux/macOS 21

4.7 Das Fenster „Über“ 21

4.8 Erstellen der FOMOD-Dateien ... 21
4.9 Übersetzungseditor ... 21
4.10 XML-Editor ... 23
4.11 Schriftartenverwaltung ... 24
4.12 Zwei FOMODs zusammenführen ... 25
4.13 Exportieren des Distributionsarchivs ... 25
4.14 Das Fenster „Eigenschaften“ ... 25
4.15 Befehlszeilenmodus (CLI) ... 27
4.16 Nach Updates suchen ... 27
5. FOMOD-Tag-Referenz ... 28
5.1 Die Datei info.xml ... 28
5.2 Die Datei ModuleConfig.xml ... 28
Stamm und Kopfzeile ... 28
Dateien und Ordner ... 29
Schritte und Gruppen ... 29
Optionen (Plugins) ... 29
Abhängigkeiten und Bedingungen ... 30
6. Die Datei ModuleConfig.xml im Detail ... 31
6.1 Wie der Installer die Datei liest ... 31
6.2 Die Wurzel und der Header ... 31
6.3 Dateien installieren ... 31
6.4 Schritte, Gruppen und Optionen ... 32
6.5 Der Typ einer Option und ihr Verhalten ... 33
6.6 Flags, Abhängigkeiten und bedingte Installationen ... 33
7. Beispiele für ModuleConfig.xml ... 35
7.1 Minimalbeispiel — erforderliche Dateien ... 35
7.2 Exklusive Auswahl — SelectExactlyOne ... 35
7.3 Bedingte Installation — Flags ... 35
7.4 Eine Seite, mehrere Schritte — SelectAny + Flags ... 36
7.5 Preset mit anpassbaren Standardwerten ... 38
7.6 Einleitungs- und Abschlussseiten ... 40
8. Version 2 — neue Funktionen ... 41
8.1 Erkennung von Zielkonflikten ... 41
8.2 7z-Export ... 41
8.3 Wiederverwendbare Vorlagen ... 41
8.4 Neu aus Ordner ... 41
8.5 Dateizuweisung per Drag-and-Drop ... 41
8.6 FOMOD-Vergleich ... 41
8.7 Visueller Bedingungseditor ... 42

8.8 Bildprüfungen und -optimierung 42
8.9 Ergänzungen der Befehlszeile 42
8.10 Export und Validierung im Hintergrund 42
8.11 Absturzwiederherstellung 42
8.12 Eine übersichtlichere Benutzeroberfläche 42
8.13 Ein vorhandenes FOMOD übersetzen 43
8.14 Ein Mod-Archiv öffnen 44
8.15 Importtreue 44
8.16 Sicherungen und Wiederherstellung 44
8.17 Master als automatische Bedingungen 45
8.18 Light-Plugins (ESL) und der Plugin-Bericht 45
8.19 Reichhaltigerer Simulator und Szenarien 45
8.20 Installationsgröße pro Option 46
8.21 Nexus-Beschreibungsgenerator 46
8.22 Ein bearbeitbarer Bedingungseditor 46
8.23 Projekt-Texttabelle 47
8.24 BSA- und BA2-Archive lesen 47
8.25 Verschachtelte Bedingungsgruppen und Versionsbedingungen 48
8.26 Spiele und ihre Kategorien bearbeiten 49
8.27 Baumfarben 49
8.28 Dateidialoge merken sich, wo Sie waren 49
8.29 Preset-Assistent 49
Anhänge 50
Anhang A — Die Datei Config.ini 50
Anhang B — Tastaturkürzel 51
Anhang C — Glossar 52
Anhang D — Vollständiges FOMOD-Beispiel 54
D.1 info.xml 54
D.2 ModuleConfig.xml 54
Anhang E — Lizenz und Danksagungen 58

1. Überblick

1.1 Was ist XIMOD Architect?

XIMOD Architect ist ein plattformübergreifendes Tool zur Erstellung von FOMOD-Installationsprogrammen für Bethesda-Spiel-Mods (Skyrim, Fallout, Starfield, Oblivion, Morrowind usw.). Damit können Sie visuell einen vollständigen Installationsassistenten erstellen – Schritte, Optionsgruppen, Optionen, bedingte Dateien – und anschließend automatisch die standardisierten XML-Dateien generieren, die von Mod-Managern interpretiert werden können.

Die Anwendung unterstützt elf Spiele, die in der Datei Categories.json definiert sind: die drei Editionen von Skyrim (Legendary, Special Edition und VR), Fallout 3, Fallout: New Vegas, Fallout 4 und Fallout 4 VR, Oblivion und Oblivion Remastered, Morrowind sowie Starfield. Da es sich um eine externe und bearbeitbare Datei handelt, lässt sich die Liste erweitern, ohne dass die Anwendung neu kompiliert werden muss.

Die Software ist eine Neuprogrammierung des ursprünglichen FOMODCreationTool in der Programmiersprache Rust. Diese Überarbeitung sorgt für eine bessere Leistung, ein identisches Verhalten unter Windows, Linux und macOS sowie eine moderne Benutzeroberfläche. XIMOD Architect zeichnet sich zudem durch sein sehr fortschrittliches Internationalisierungssystem aus: Die Benutzeroberfläche kann in eine Vielzahl von Sprachen übersetzt werden, wobei die für das jeweilige Schriftsystem geeigneten Schriftarten dynamisch geladen werden.

Die wichtigsten Funktionen sind:

- Erstellung vollständiger FOMOD-Installationspakete;
- mehrstufige Installationsassistenten;
- Optionsgruppen mit verschiedenen Auswahltypen (genau eines, mindestens eines, beliebig viele usw.);
- bedingte Dateiinstallation basierend auf den Auswahlentscheidungen des Benutzers;
- Abhängigkeitsmuster, die den Typ einer Option bestimmen;
- ein System von „Flags“ (Bedingungsflags), das den Ablauf der Installation steuert;
- benutzerdefinierte Skripte, die vor oder nach dem Speichern ausgeführt werden;
- eine mehrsprachige Benutzeroberfläche und dynamische Schriftarten;
- ein nativer, transparenter Startbildschirm.

1.2 Das FOMOD-Format

FOMOD („Fallout Mod“) ist das Standardformat für Mod-Installationsprogramme für Bethesda-Spiele. Ein FOMOD-Installationsprogramm basiert auf zwei XML-Dateien, die in einem Unterordner namens fomod gespeichert sind:

- info.xml — die Mod-Metadaten: Name, Autor, Version, Website, Beschreibung, Kategorie.
- `ModuleConfig.xml — the installation configuration: steps, groups, options, files, conditions.`

Die von XIMOD Architect erstellten Dateien sind mit den wichtigsten Mod-Managern kompatibel:

- Mod Organizer 2 (MO2);
- Vortex;
- Nexus Mod Manager (NMM).

> Hinweis — Der Name des fomod-Ordners sowie die Namen info.xml und ModuleConfig.xml sind durch die Spezifikation vorgegeben und werden von den Mod-Managern unverändert erkannt. Sie dürfen niemals umbenannt werden.

1.3 Installation und Anforderungen

Systemanforderungen (Richtwerte):

Element	Empfehlung
System	Windows 10 oder höher, Linux (aktuelle glibc) oder macOS 11 oder höher.
Bildschirm	Grafikkarte, die mit OpenGL 3.3 kompatibel ist (eguis „glow“-Renderer).
Arbeitsspeicher	Einige hundert MB — die Anwendung ist ressourcenschonend.
Speicherplatz	Einige Dutzend MB, abhängig von den Noto-Schriftarten und den in assets/ bereitgestellten Flags.

Windows

Legen Sie die ausführbare Datei ximod-architect.exe, den Ordner assets (Daten, Sprachpakete, Schriftarten, Bilder, Symbole) und optional die Datei assets/images/splash.png für den Startbildschirm in denselben Ordner. Das Anwendungssymbol wird beim Kompilieren in die ausführbare Datei eingebettet. Führen Sie anschließend ximod-architect.exe aus.

Linux

Ein Installationsskript ist im Verzeichnis packaging/linux enthalten:

```
# System-wide installation (requires sudo)
sudo ./packaging/linux/install.sh

# Installation for the current user
./packaging/linux/install.sh --user
```

Das Erstellen aus dem Quellcode erfordert Rust 1.98.1 oder höher und unter Linux die folgenden Entwicklungsbibliotheken: libgtk-3-dev, libxcb-render0-dev, libxcb-shape0-dev, libxcb-xfixes0-dev, libspeechd-dev, libxkbcommon-dev, libssl-dev und libx11-dev.

macOS

Ein Skript erstellt ein zur Verteilung bereites Anwendungsbundle (.app):

```
./packaging/macos/create-bundle.sh
```

Das Bundle „XIMOD Architect.app“ wird im Ordner dist erstellt. Anschließend kann mit dem Dienstprogramm hdiutil ein Datenträgerabbild (.dmg) erzeugt werden.

1.4 Dateistruktur

Der Dateibaum, der die ausführbare Datei begleitet, sieht wie folgt aus:

```
ximod-architect(.exe)            → executable
Config.ini                       → configuration (created on first launch)
assets/
   data/
      Categories.json            → games and Nexus categories
      Languages.json             → 6,777 languages (endonym, ISO codes, font)
      Countries.json             → 244 countries (names, flag, official languages)
      CountryLanguages.json      → languages spoken by country
   locales/                      → translations (one folder per language)
      eng/main.ftl
      fra/main.ftl
      …                          → 32 languages provided
   fonts/                        → Noto fonts (one per writing system)
   icons/                        → application icons
   images/
      splash.png                 → splash screen
      svg/                       → country flags (244 SVG files)
```

2. Erste Schritte

2.1 Starten der Anwendung

Beim Start zeigt XIMOD Architect zunächst einen nativen, transparenten Startbildschirm mit einem Überblendeffekt an, dessen Dauer einstellbar ist (standardmäßig zwei Sekunden; der Bildschirm kann deaktiviert werden). Anschließend öffnet sich das Hauptfenster, zentriert auf dem primären Bildschirm. Seine anfängliche Größe stammt aus der Konfigurationsdatei (standardmäßig 1280 × 800) und kann 800 × 600 nicht unterschreiten. Die Titelleiste zeigt den Namen der Anwendung gefolgt von ihrer Versionsnummer.

2.2 Übersicht über die Benutzeroberfläche

Das Hauptfenster besteht aus fünf Bereichen:

- die Menüleiste oben — Datei, Bearbeiten, Projekt, Werkzeuge und Hilfe — und direkt darunter die Symbolleiste mit den alltäglichen Aktionen (Neu, Öffnen, Speichern, Rückgängig, Wiederholen, Validieren, Vorschau, Exportieren); siehe Kapitel 3;
- der Projektbaum auf der linken Seite: Mod-Informationen, dann die Installationsschritte mit ihren Gruppen und Optionen, die erforderlichen Dateien und die bedingten Installationen. Klicken Sie auf einen Knoten, um ihn zu bearbeiten, klicken Sie mit der rechten Maustaste darauf für seine Aktionen (hinzufügen, duplizieren, verschieben, löschen, als Vorlage speichern), und ziehen Sie eine Option auf eine andere Gruppe, eine Gruppe auf einen anderen Schritt oder einen Schritt vor einen anderen Schritt, um ihn zu verschieben; die +-Schaltflächen neben „Installationsschritte“ und „Bedingte Installationen“ fügen direkt ein Element hinzu;
- der Inspektor in der Mitte, der den Editor des ausgewählten Knotens anzeigt: die Mod-Informationen, einen Schritt (Name, Sichtbarkeitsbedingungen, Seitenziel, Liste seiner Gruppen), eine Gruppe (Name, Auswahltyp, Gruppenziel, Liste ihrer Optionen), eine Option (Beschreibung, Bild, Typ, Flags, Abhängigkeitsmuster, Dateien), die erforderlichen Dateien oder einen bedingten Satz; eine Brotkrumenleiste oben im Inspektor ermöglicht es, eine Ebene nach oben zu gehen;
- das Bedienfeld „Probleme“ am unteren Rand, das nach Projekt → FOMOD validieren angezeigt wird: Jeder Befund wird mit seinem Schweregrad aufgeführt, und ein Klick darauf wählt den betreffenden Knoten im Baum aus; der Baum selbst zeigt einen kleinen Fehler-/Warnungszähler neben den betreffenden Knoten an;
- die Statusleiste ganz unten: links die letzte Meldung (beim Start: „Bereit“); rechts die Anzahl der Schritte und Optionen sowie, wenn ein Projekt ungespeicherte Änderungen enthält, ein Punkt gefolgt von „Geändert“. Erfolge, Warnungen und Fehler erscheinen außerdem als Benachrichtigungen in der oberen rechten Ecke.

Das Erscheinungsbild der Benutzeroberfläche (dunkles, helles oder System-Design, Schriftgröße, Sprache) wird im Fenster „Einstellungen“ festgelegt, das in § 4.5 beschrieben wird.

Im Baum wählt ein Klick einen Knoten aus und zeigt ihn im Inspektor an; ein Doppelklick auf einen Schritt oder eine Gruppe tut dasselbe und öffnet ihn. Das Rechtsklick-Menü eines Schritts bietet Aufklappen (Alle, Ausgewählte, ab Ausgewähltem — der Schritt und alles darunter) und Zuklappen (dieselben drei Möglichkeiten), und das Menü einer Gruppe bietet Aufklappen und Zuklappen (Alle, Ausgewählte), vor den üblichen Befehlen Hinzufügen / Duplizieren / Verschieben / Löschen. Jede Registerkarte behält ihr eigenes Bedienfeld „Probleme“: Die angezeigten Befunde gehören immer zum FOMOD, das in der aktiven Registerkarte geladen ist, und eine in einer Registerkarte gestartete

Validierung wird in dieser Registerkarte abgeschlossen, auch wenn Sie zwischenzeitlich zu einer anderen wechseln.

Der Baum kann auch über die Tastatur gesteuert werden, solange kein Textfeld den Fokus hat: ↑ und ↓ bewegen einen Cursorring von Zeile zu Zeile (er beginnt beim ausgewählten Knoten), Enter wählt die Zeile darunter aus, → klappt einen geschlossenen Schritt oder eine geschlossene Gruppe auf (und wechselt bei einem geöffneten zu seinem ersten Kind), ← klappt einen geöffneten zu (und geht bei einer geschlossenen Zeile oder einer Option zurück zum übergeordneten Element). Der Ring wird beim Bewegen in den sichtbaren Bereich gescrollt.

2.3 Ein neues Projekt erstellen

Die Erstellung eines Installationsprogramms erfolgt im Allgemeinen in den folgenden Schritten:

- Neues Projekt — Menü Datei → Neu, dann das Stammverzeichnis auswählen, das die Dateien Ihres Mods enthält (Schaltfläche „Durchsuchen…“ auf der Registerkarte „Mod-Info“).
- Informationen ausfüllen — Geben Sie auf der Registerkarte „Mod-Info“ den Mod-Namen, den Autor, die Version, das Spiel, die Kategorie, die Website, das Kopfbild und die Beschreibung ein.
- Den Assistenten aufbauen — Fügen Sie auf der Registerkarte „Installationsschritte“ Schritte, dann Optionsgruppen und dann Optionen hinzu.
- Die Dateien zuordnen — Fügen Sie für jede Option die zu installierenden Dateien oder Ordner hinzu und legen Sie anschließend bei Bedarf ihr Ziel in der Tabelle fest (es ist standardmäßig leer, was die Installation im Stammverzeichnis des Data-Ordners des Spiels bewirkt).
- Die Bedingungen festlegen — Verwenden Sie Flags und bedingte Installationen, um die Installation an die Auswahl des Benutzers anzupassen.
- Speichern — Menü Datei → Speichern (Ctrl+S): XIMOD Architect erstellt den Ordner fomod und schreibt info.xml und ModuleConfig.xml hinein.

2.4 Projekt öffnen und speichern

Beim Speichern wird immer der Unterordner fomod unter dem Stammverzeichnis erzeugt und anschließend werden die beiden XML-Dateien in UTF-8 hineingeschrieben. Der Befehl „Speichern“ bleibt deaktiviert, solange kein Stammverzeichnis festgelegt wurde.

Es werden zwei Öffnungsmodi angeboten. „Ordner öffnen…“ fragt nach dem Stammverzeichnis und lädt das Projekt aus seinem Unterordner fomod. „Datei öffnen…“ fragt direkt nach einer XML-Datei: Das Stammverzeichnis wird dann abgeleitet (es wird angenommen, dass sich die Datei in einem Unterordner fomod befindet). Beim Lesen erkennt XIMOD Architect automatisch die Dateikodierung (UTF-8, mit oder ohne Byte Order Mark, sowie UTF-16), was es ermöglicht, von anderen Werkzeugen erstellte Installationsprogramme erneut zu öffnen.

Jedes erfolgreich geöffnete Projekt wird am Anfang der Liste der zuletzt verwendeten Dateien hinzugefügt, die über Datei → Zuletzt verwendet zugänglich ist und auf der Registerkarte „Zuletzt verwendete Dateien“ der Einstellungen verwaltet werden kann.

2.5 Erster Start

Beim allerersten Start startet XIMOD Architect auf Englisch und öffnet automatisch das Fenster „Einstellungen“, damit der Benutzer sein Land und dann seine Sprache auswählen kann.

Dieses Verhalten wird durch den Schlüssel FirstStart in der Datei Config.ini gesteuert: Er ist 0, solange die anfängliche Konfiguration nicht gespeichert wurde, und wechselt auf 1, sobald zum ersten Mal auf „Speichern“ geklickt wird. Nachfolgende Starts erfolgen dann direkt in der gewählten Sprache.

Wenn der Benutzer das Fenster mit „Abbrechen“ schließt, bleibt der Schlüssel auf 0 und das Fenster öffnet sich beim nächsten Start erneut.

2.6 Freie Fenster und Tastaturnavigation

Mehrere Werkzeuge öffnen sich in freien (unabhängigen) Fenstern: die „Länderauswahl“, das Fenster „Eigenschaften“, der Übersetzungseditor und der XML-Editor. Jedes davon ist frei beweglich und in der Größe veränderbar, auch auf einem zweiten Bildschirm, und arbeitet neben dem Hauptfenster.

Zentrierung und Persistenz — Beim ersten Öffnen wird ein freies Fenster auf dem XIMOD-Hauptfenster zentriert. Wenn der Benutzer es verschiebt oder in der Größe verändert, werden seine Position und Größe beim Schließen (und beim Beenden der Anwendung) gespeichert und beim nächsten Öffnen wiederhergestellt. Diese Werte werden in Config.ini im Abschnitt [WindowPositions] gespeichert (siehe Anhang A). Das Hauptfenster selbst merkt sich seine Größe, aber nicht seine Position: Es bleibt bei jedem Start zentriert.

Schließen mit Esc — Die Esc-Taste schließt das aktive freie Fenster sowie die modalen Dialoge des Hauptfensters (Einstellungen, Über, Bestätigung, Prüfbericht, Skripte); bei einer Bestätigung wirkt Esc wie „Abbrechen“ und führt die Aktion niemals aus. Zwei Ausnahmen: Der Übersetzungseditor (um zu vermeiden, dass er während der Eingabe versehentlich geschlossen wird) und das XIMOD-Hauptfenster schließen sich nicht mit Esc.

Tastaturnavigation — Die Listen im Fenster „Eigenschaften“, die Tabelle des Übersetzungseditors und das Flaggenraster der Länderauswahl lassen sich vollständig über die Tastatur bedienen: Pfeiltasten (und der Ziffernblock), Page Up/Page Down, Home/End, Enter und Tab/Shift+Tab. Die für jedes Fenster spezifischen Einzelheiten werden in den jeweiligen Abschnitten beschrieben und in Anhang B zusammengefasst.

Filter löschen — Jedes Feld „Filter:“ hat rechts davon ein Löschsymbol (rotes Kreuz), das den Filter mit einem einzigen Klick leert.

3. Die Menüs

Die Menüleiste enthält fünf Menüs: Datei, Bearbeiten, Projekt, Werkzeuge und Hilfe. Darunter bietet eine Symbolleiste Zugriff mit einem Klick auf die alltäglichen Aktionen (Neu, Öffnen, Speichern, Validieren, Vorschau, Exportieren). Die angezeigten Tastenkürzel sind auch global aktiv (die Ctrl-Taste entspricht Cmd unter macOS); sie werden nur ignoriert, während ein modaler Dialog geöffnet ist. Modale Dialoge dimmen den Rest des Fensters; Enter löst ihre primäre Schaltfläche aus und Esc bricht ab.

3.1 Menü „Datei“

Befehl	Tastenkürzel	Beschreibung
Neu	Ctrl+N	Öffnet ein neues leeres Projekt in einer neuen Registerkarte (die aktuelle Registerkarte wird nur ersetzt, wenn sie noch unberührt ist).
Neu aus Ordner…	—	Durchsucht einen Mod-Ordner und schlägt ein Grundgerüst vor, bei dem jeder Unterordner zu einer Option wird. Siehe § 8.4.
Ordner öffnen…	Ctrl+O	Wählt ein Stammverzeichnis aus und lädt das Projekt aus seinem Unterordner fomod.
Datei öffnen…	Ctrl+Shift+O	Wählt direkt eine info.xml oder ModuleConfig.xml aus; das Stammverzeichnis wird abgeleitet. Eine Datei, die sich nicht in einem fomod-Ordner befindet, wird abgelehnt.
Archiv öffnen…	—	Entpackt ein .zip- oder .7z-Mod-Archiv (wie von Nexus heruntergeladen) in den Arbeitsordner des Konfigurationsverzeichnisses und öffnet dann sein FOMOD. Ein bereits entpacktes Archiv wird wiederverwendet. .rar-Archive werden abgelehnt: Entpacken Sie sie zuerst mit 7-Zip. Das Ablegen eines Archivs auf dem Fenster bewirkt dasselbe. Siehe § 8.14.
Speichern	Ctrl+S	Erzeugt fomod/info.xml und fomod/ModuleConfig.xml. Wenn das Projekt noch kein Stammverzeichnis hat, wird zuerst nach einem Ordner gefragt.
Speichern unter…	Ctrl+Shift+S	Wählt ein (neues) Stammverzeichnis für das Projekt aus und speichert es dort.
Sicherung wiederherstellen…	—	Listet die früheren Versionen der FOMOD-XML-Dateien auf, die unter fomod/backups aufbewahrt werden, mit der Anzahl der Unterschiede zum aktuellen Projekt; jede kann verglichen oder in den Editor wiederhergestellt werden. Deaktiviert, wenn das Projekt noch keine Sicherung hat. Siehe § 8.16.
FOMOD zusammenführen…	—	Fügt die Schritte, erforderlichen Dateien und bedingten Installationen eines anderen FOMOD zum aktuellen Projekt hinzu. Deaktiviert, solange kein Stammverzeichnis festgelegt ist. Siehe § 4.12.

Befehl	Tastenkürzel	Beschreibung
Distributionsarchiv exportieren…	—	Schreibt das FOMOD und komprimiert dann das gesamte Stammverzeichnis in ein veröffentlichungsbereites .zip- oder .7z-Archiv, im Hintergrund mit einem Fortschrittsfenster. Siehe § 4.13 und § 8.10.
FOMOD schließen	—	Schließt die aktive Registerkarte und fragt zuvor nach, wenn sie ungespeicherte Änderungen enthält.
Alle FOMODs schließen	—	Schließt jede Registerkarte und fragt zuvor nach, wenn eine ungespeicherte Änderungen enthält.
Zuletzt verwendet	—	Untermenü, das die zuletzt geöffneten Projekte auflistet. Zeigt „(leer)“ an, wenn die Liste leer ist.
Beenden	Ctrl+Q	Schließt die Anwendung. Ungespeicherte Projekte werden aufgelistet und können zuvor gespeichert werden; die Konfiguration wird beim Beenden gespeichert.

3.2 Menü „Bearbeiten“

Befehl	Tastenkürzel	Beschreibung
Rückgängig	Ctrl+Z	Macht die letzte Änderung am Projekt rückgängig (Tastenanschläge im selben Feld werden zusammengefasst). Deaktiviert, wenn es nichts rückgängig zu machen gibt.
Wiederholen	Ctrl+Y	Wendet die letzte rückgängig gemachte Änderung erneut an (Ctrl+Shift+Z funktioniert ebenfalls).

3.3 Menü „Projekt“

Alles, was auf das gerade bearbeitete FOMOD wirkt.

Befehl	Tastenkürzel	Beschreibung
FOMOD validieren	—	Überprüft das Projekt und die Konformität von ModuleConfig.xml mit dem Schema ModConfig 5.0, verifiziert die referenzierten Dateien, Zielkonflikte und Bilder (im Hintergrund) und zeigt dann einen detaillierten Bericht an. Siehe § 4.8.
Installationsprogramm-Vorschau…	—	Simuliert das Installationsprogramm so, wie es ein Mod-Manager anzeigen würde. Siehe § 4.11.
Bedingungseditor…	—	Jedes Bedingungsflag mit Angabe, wo es gesetzt und verwendet wird (anklickbar), Umbenennen oder Entfernen eines Flags überall sowie ein Bedingungsersteller für den ausgewählten Knoten. Siehe § 8.7 und § 8.22.
FOMOD übersetzen…	—	Öffnet das Fenster, das ein vorhandenes Installationsprogramm (das aktive Projekt, einen Ordner oder ein Archiv) übersetzt, ohne seine XML-Struktur zu verändern. Siehe § 8.13.

Befehl	Tastenkürzel	Beschreibung
Vergleichen mit…	—	Vergleicht das Projekt mit einem anderen FOMOD und listet auf, was sich geändert hat. Siehe § 8.6.
Vorlagen…	—	Speichert den ausgewählten Schritt als wiederverwendbare Vorlage oder fügt eine gespeicherte Vorlage ein. Siehe § 8.3.
Preset-Assistent…	—	Erzeugt im aktuellen Projekt ein Installationsprogramm nach dem Muster „Preset + Anpassen“: eine Seite für den Installationsmodus mit den Presets, bei Bedarf angezeigte Funktionsseiten, dynamische Standardwerte und bedingte Installationen. Siehe § 7.5 und § 8.29.
XML-Editor	—	Untermenü, das Zugriff auf info.xml und ModuleConfig.xml bietet. Siehe § 4.10.

3.4 Menü „Werkzeuge“

Einstellungen und Dienstprogramme, die nicht an ein bestimmtes Projekt gebunden sind.

Befehl	Tastenkürzel	Beschreibung
Einstellungen…	Ctrl+,	Öffnet das Fenster „Einstellungen“ (Land, Sprache, Design, Schriftart usw.). Siehe § 4.5.
Baumfarben…	—	Legt für jedes Design eine Farbe pro Zeilenart des Projektbaums fest (Schritte, Gruppen, Optionen…). Siehe § 8.27.
Skript vor dem Speichern…	—	Bearbeitet das vor dem Speichern ausgeführte Skript. Siehe § 4.6.
Skript nach dem Speichern…	—	Bearbeitet das nach dem Speichern ausgeführte Skript. Siehe § 4.6.
Plugin-Bericht…	—	Tabelle jeder vom Projekt installierten Plugin-Datei (.esp, .esm, .esl): Art, Light-Flag, Master (fehlende hervorgehoben), neue Datensätze im Vergleich zum Light-Plugin-Limit und ESL-Eignung. Siehe § 8.18.
Archivinhalt…	—	Listet die in einem Bethesda-Archiv .bsa oder .ba2 gepackten Dateien auf (Format, Größen, Komprimierung) mit einem Suchfeld. Siehe § 8.24.
Nexus-Beschreibung…	—	Generiert den Text der Nexus-Mods-Seite (BBCode oder Markdown) aus dem Projekt: Voraussetzungen, Installationsoptionen, Installationshinweis und Änderungsprotokoll gegenüber einer früheren Version; kopieren oder speichern. Siehe § 8.21.
Projekt-Texte…	—	Tabelle jedes Namens und jeder Beschreibung des Projekts, mit Suche, Suchen/Ersetzen und Duplikaterkennung; die Texte werden direkt bearbeitet. Siehe § 8.23.
Benutzeroberfläche übersetzen…	—	Öffnet den Editor zur Übersetzung der Benutzeroberfläche. Siehe § 4.9.

Befehl	Tastenkürzel	Beschreibung
Eigenschaften…	—	Öffnet den schreibgeschützten Explorer der Länder-/Sprachdatenbank. Siehe § 4.14.
Spiele / Kategorien…	—	Öffnet den Spiele-Editor: Hinzufügen, Korrigieren oder Entfernen eines Spiels und seiner Nexus-Kategorien (Name, slug, Basis-Master, Grenze für Light-Plugins), ohne Categories.json von Hand zu bearbeiten. Siehe § 8.26.

3.5 Menü „Hilfe“

Befehl	Tastenkürzel	Beschreibung
Benutzerhandbuch	F1	Öffnet dieses Handbuch (PDF) in der Sprache der Benutzeroberfläche aus dem mit der Anwendung ausgelieferten Ordner Manuals.
Nach Updates suchen…	—	Fragt die neueste veröffentlichte Version ab und bietet die Download-Links an.
Über	—	Zeigt das Fenster „Über“ an (Name, Version, Lizenz). Siehe § 4.7.

4. Funktionen im Detail

4.1 Die Registerkarte „Mod-Info“

Diese Registerkarte fasst die Mod-Metadaten und die Auswahl des Stammverzeichnisses zusammen.

Arbeitsbereich — Die Zeile „Stammverzeichnis:“ zeigt den Arbeitsordner an (oder „(Verzeichnis auswählen)“, wenn es nicht festgelegt ist). Mit der Schaltfläche „Durchsuchen…“ können Sie es auswählen.

Die Metadatenfelder sind dann die folgenden:

Feld	Rolle
Mod-Name	Angezeigter Name des Installationsprogramms (erforderlich).
Autor	Name des Mod-Autors.
Version	Versionsnummer (standardmäßig 1.0.0 für ein neues Projekt).
Spielname	Zielspiel, ausgewählt aus einer Liste, die aus Categories.json stammt.
Kategorie	Kategorie des Mods. Hängt vom gewählten Spiel ab; andernfalls wird eine Standardliste angeboten (Animation, Rüstung, Audio, Texturen, Waffen usw.).
Website	Adresse der Mod-Seite.
Kopfbild	Bild, das oben im Installationsprogramm angezeigt wird. Schaltflächen „Durchsuchen…“ und „Löschen“; eine Vorschau wird angezeigt. Der Pfad wird relativ zum Stammverzeichnis gespeichert.
Beschreibung	Vorstellungstext des Mods (mehrzeiliger Bereich).

> Hinweis — Jede Änderung an einem Feld markiert das Projekt in der Statusleiste als „Geändert“.

Nexus-Link — Neben dem Auswahlfeld „Spielname“ öffnet eine Schaltfläche „Nexus ↗“ direkt im Browser die Nexus-Mods-Seite des gewählten Spiels (aus seinem „slug“ aufgebaut). Sie ist nur aktiv, wenn das ausgewählte Spiel einen slug hat.

Die Listen „Spielname“ und „Kategorie“ stammen aus der externen Datei assets/data/Categories.json, die ohne Neukompilierung bearbeitet werden kann. Jedes Spiel wird dort durch einen Bezeichner, einen Namen, einen Nexus-„slug“ und seine Kategorienliste beschrieben:

```
{
  "games": {
    "skyrimSpecialEdition": {
      "name": "The Elder Scrolls V: Skyrim Special Edition",
      "nexusSlug": "skyrimspecialedition",
      "categories": ["Animation", "Armour", "Audio", "…"]
    }
  }
}
```

4.2 Die Registerkarte „Installationsschritte“

Dies ist das Herzstück des Installationsassistenten. Er basiert auf einer dreistufigen Hierarchie: Schritte → Gruppen → Optionen, was genau der Form des Projektbaums auf der linken Seite entspricht. Fügen Sie einen Schritt mit der +-Schaltfläche neben „Installationsschritte“ hinzu (standardmäßig „Schritt N“ benannt) oder über den leeren Inspektor; wählen Sie einen Schritt, eine Gruppe oder eine Option im

Baum aus, um sie in der Mitte zu bearbeiten. Der Schritt-Editor zeigt den Namen (bearbeitbar), die Schaltfläche „Schritt löschen“ (die einzige Löschung, die eine Bestätigung verlangt: alles andere kann mit Ctrl+Z rückgängig gemacht werden), die Sichtbarkeitsbedingungen, das Seitenziel und die Liste der Gruppen mit einer Schaltfläche „Gruppe hinzufügen“; der Gruppen-Editor zeigt den Namen, den Auswahltyp, das Gruppenziel und die Liste der Optionen mit einer Schaltfläche „Option hinzufügen“; der Options-Editor wird weiter unten beschrieben.

Gruppen und Auswahltypen

Jede Gruppe hat einen Namen und einen Auswahltyp, der bestimmt, wie viele Optionen der Endbenutzer ankreuzen kann. Die fünf verfügbaren Typen sind:

Typ	Bedeutung
Genau eines (erforderlich) — SelectExactlyOne	Der Benutzer muss genau eine Option wählen.
Höchstens eines — SelectAtMostOne	Höchstens eine Option (oder keine).
Beliebig viele — SelectAny	Beliebig viele Optionen (Standardwert).
Alle (keine Auswahl) — SelectAll	Alle Optionen sind ausgewählt.
Mindestens eines — SelectAtLeastOne	Mindestens eine Option.

Die Schaltfläche „Gruppe hinzufügen“ des Schritt-Editors (oder das Rechtsklick-Menü des Schritts) erstellt eine Gruppe, standardmäßig vom Typ „Beliebig viele“; „Löschen“ im Rechtsklick-Menü der Gruppe oder im Gruppen-Editor entfernt sie.

Neu anordnen — Die Anzeigereihenfolge wird angepasst, ohne alles neu zu erstellen: Ziehen Sie einen Knoten im Baum und legen Sie ihn dort ab, wo er hingehört (eine Option vor eine andere Option oder auf eine Gruppe, eine Gruppe vor eine andere Gruppe oder auf einen Schritt, einen Schritt vor einen anderen Schritt), oder verwenden Sie „Nach oben“ / „Nach unten“ im Rechtsklick-Menü des Knotens. „Duplizieren“ kopiert einen Schritt, eine Gruppe oder eine Option mit allem, was sie enthält. Jede Änderung kann mit Bearbeiten → Rückgängig (Ctrl+Z) rückgängig gemacht und mit Bearbeiten → Wiederholen (Ctrl+Y) wiederholt werden; aufeinanderfolgende Tastenanschläge in einem Feld zählen als ein Schritt.

Optionen und Standardtypen

Eine Option (ein <plugin>-Element im FOMOD-Format) ist eine dem Benutzer angebotene Auswahl. Sie hat einen Namen, eine Beschreibung, ein Bild (Vorschau, Schaltflächen „Durchsuchen…“ und „Löschen“) und einen Standardtyp:

Typ	Bedeutung
Optional — Optional	Optionale Option (Standard).
Erforderlich — Required	Option, die immer installiert wird.
Empfohlen — Recommended	Empfohlene, aber optionale Option.
Nicht verwendbar — NotUsable	Nicht verwendbare Option.

Typ	Bedeutung
Könnte verwendbar sein — CouldBeUsable	Je nach Bedingungen potenziell verwendbare Option.

Plugin-Dateien

Der Abschnitt „Quelle“ zeigt in einer vierspaltigen Tabelle (Typ, Quelle, Ziel, Priorität) die von der Option installierten Dateien und Ordner an. Die Schaltflächen „Datei hinzufügen“, „Ordner hinzufügen“ und „Entfernen“ verwalten die Liste. Die Spalte „Ziel“ ist bearbeitbar: Klicken Sie auf eine Zelle und geben Sie den Pfad relativ zum Data-Ordner des Spiels ein, unter dem das Element installiert werden soll. Sie ist standardmäßig leer, was die Datei — oder den Inhalt des Ordners — im Stammverzeichnis von Data installiert; dies ist die richtige Wahl, wenn der Quellordner die Struktur des Spiels bereits wiedergibt (zum Beispiel ein Ordner, der selbst meshes, textures, SKSE usw. enthält). Geben Sie nur dann einen expliziten Pfad an, wenn das Ziel abweicht, zum Beispiel SKSE\Plugins\MyMod.ini. Die Dateien müssen sich unter dem Stammverzeichnis befinden. Sie können Dateien oder Ordner auch direkt aus Ihrem Dateimanager (Windows-Explorer, Finder…) auf das XIMOD-Fenster ziehen, während die Option ausgewählt ist: Sie werden sofort zu ihrer Quelltabelle hinzugefügt (siehe § 8.5). Der Dialog öffnet sich im Ordner Ihrer letzten Auswahl, sodass benachbarte Ordner nur einen Klick entfernt sind (§ 8.28).

Gruppen-/Seitenziel

Um mehrere Optionen an denselben Ort zu senden, ohne jede Datei einzeln zu bearbeiten, verwenden Sie die Steuerelemente für die Massenzuweisung des Ziels. Im Editor einer Gruppe legen das Feld „Ziel für die gesamte Gruppe“ und seine Schaltfläche „Auf alle Optionen dieser Gruppe anwenden“ ein Installationsziel für jede Datei jeder Option in dieser Gruppe auf einmal fest. Auf Schrittebene tut das einklappbare Bedienfeld „Installationsziel (gesamte Seite)“ dasselbe für jede Option auf der Seite über alle ihre Gruppen hinweg. Eine Statusmeldung bestätigt, wie viele Dateien aktualisiert wurden. Die Schaltfläche bleibt deaktiviert, bis die Optionen tatsächlich Dateien enthalten, und Sie können anschließend jedes einzelne Ziel in der Quelltabelle der Option noch feinabstimmen.

Bedingungsflags

Der Abschnitt „Flag-Name:“ ermöglicht es, der Option Flags zuzuordnen (Name-Wert-Paare). Wenn eine Option vom Endbenutzer ausgewählt wird, werden ihre Flags gesetzt; sie können dann die Sichtbarkeit eines Schritts, den Typ einer Option (Abhängigkeitsmuster) oder die Installation bedingter Dateien bedingen. Die Schaltflächen „Flag hinzufügen“ und „Flag entfernen“ verwalten die Liste.

Autovervollständigung — Die Namens- und Wertfelder von Flags und Abhängigkeiten bieten während der Eingabe ein Menü mit Vorschlägen an, die aus den bereits im Projekt vorhandenen Elementen stammen, was Tippfehler vermeidet, die eine Übereinstimmung stillschweigend zerstören würden. Für eine Dateiabhängigkeit schlägt der Wert die Zustände Active / Inactive / Missing vor. Die Vorschläge sind nicht bindend: Ein neuer Wert kann weiterhin frei eingegeben werden.

Plugin-Abhängigkeiten. Unterhalb der Flags macht dieser Abschnitt den Typ einer Option dynamisch. Der Standardtyp oben gilt, wenn kein Muster passt; jedes Muster gibt der Option einen anderen Typ, wenn seine Bedingungen erfüllt sind. Verwenden Sie „Neues Muster“ / „Muster löschen“, um die Liste zu verwalten; wählen Sie für das ausgewählte Muster den Musteroperator (And / Or) und den anzuwendenden Mustertyp (Optional, Required, Recommended, NotUsable oder CouldBeUsable) und fügen Sie dann seine Bedingungen hinzu. Eine Bedingung hat einen Typ (File oder Flag), einen Datei-/Flag-Namen (mit Autovervollständigung) und, bei einer Datei, einen Zustand (Active, Inactive oder

Missing) oder, bei einem Flag, seinen erwarteten Wert. Setzen Sie zum Beispiel den Standardtyp auf NotUsable und fügen Sie ein Muster vom Typ Optional hinzu, dessen Bedingung die Datei DLCCoast.esm im Zustand Active ist, sodass die Option nur dann auswählbar wird, wenn Far Harbor installiert ist.

Sichtbarkeitsbedingungen eines Schritts. Jeder Schritt kann bedingt angezeigt werden. Unter dem Schrittnamen enthält der einklappbare Abschnitt „Sichtbarkeitsbedingungen“ die Sichtbarkeitsregel des Schritts: Legen Sie den Operator (And / Or) fest und fügen Sie File- oder Flag-Bedingungen auf dieselbe Weise hinzu. Wenn er mindestens eine Bedingung enthält, erscheint der Schritt nur dann im Assistenten, wenn sie erfüllt sind — zum Beispiel ein Patch-Schritt, der nur angezeigt wird, wenn DLCCoast.esm Active ist. Lassen Sie ihn leer, um den Schritt immer anzuzeigen.

4.3 Die Registerkarte „Erforderliche Installationen“

Diese Registerkarte listet die Dateien und Ordner auf, die systematisch installiert werden, unabhängig von der Auswahl des Benutzers. Sie bietet dieselbe vierspaltige Tabelle (Typ, Quelle, Ziel, Priorität) — mit derselben bearbeitbaren Spalte „Ziel“ — und dieselben Schaltflächen „Datei hinzufügen“, „Ordner hinzufügen“ und „Entfernen“ wie der Dateiabschnitt einer Option. Oberhalb der Tabelle legen das Feld „Ziel“ und seine Schaltfläche „Auf alle erforderlichen Dateien anwenden“ ein Installationsziel für alle erforderlichen Dateien auf einmal fest — das Gegenstück zu den Steuerelementen für Gruppen und Seiten aus § 4.2; jedes einzelne Ziel lässt sich danach weiterhin feinabstimmen.

4.4 Die Registerkarte „Bedingte Installationen“

Diese Registerkarte gruppiert Muster: Jedes Muster installiert einen Satz von Dateien nur dann, wenn seine Bedingungen erfüllt sind. Eine Reihe von Registerkarten „Muster N“ ermöglicht es, ein Muster hinzuzufügen („ ✚ “) oder zu entfernen („ ━ “).

Für das ausgewählte Muster:

- Operator — And oder Or: wie die Abhängigkeiten des Musters kombiniert werden.
- Abhängigkeiten — eine Liste von Bedingungen vom Typ Flag ([Flag] Name = Wert) oder File ([File] Name (Zustand)). Die Schaltflächen „Abhängigkeit hinzufügen“ und „Abhängigkeit entfernen“ verwalten die Liste.
- Dateien — die vierspaltige Tabelle der Dateien, die installiert werden, wenn die Bedingungen erfüllt sind. Darüber legen das Feld „Ziel“ und seine Schaltfläche „Auf alle Dateien dieses Satzes anwenden“ ein Installationsziel für alle Dateien des Musters auf einmal fest — praktisch, nachdem der Preset-Assistent (§ 8.29) die Dateien der Optionen hierher verschoben hat.

Hinweis — „Plugin-Abhängigkeiten“. Eine Plugin-Abhängigkeit ist einfach eine File-Abhängigkeit, deren Datei ein Spiel-Plugin (.esm/.esp) ist. Geben Sie im Namensfeld den Dateinamen des Plugins so ein, wie er im Spiel erscheint (zum Beispiel Fallout4.esm, DLCCoast.esm oder die .esp eines anderen Mods), nicht den Anzeigenamen des Mods. Wählen Sie den Zustand: Active (Plugin vorhanden und aktiviert — die übliche Wahl), Inactive (vorhanden, aber deaktiviert) oder Missing (abwesend). Eine File-Abhängigkeit kann an drei Stellen verwendet werden: um Dateien automatisch zu installieren (Registerkarte „Bedingte Installationen“), um einen Assistentenschritt anzuzeigen oder auszublenden (eine visible-Bedingung eines Schritts) oder um den Zustand einer Option zu ändern — zum Beispiel sie auf Recommended zu setzen, wenn ein DLC aktiv ist, oder auf NotUsable, wenn ein erforderliches Plugin fehlt.

4.5 Das Fenster „Einstellungen“

Zugänglich über Werkzeuge → Einstellungen (Ctrl+,). Es hat zwei Registerkarten: Allgemein und Zuletzt verwendete Dateien. Die Einstellungen werden in Config.ini gespeichert.

Die Registerkarte „Allgemein“ ist in zwei Spalten gegliedert.

Linke Spalte — die Landesflagge und ihr Name:

Element	Rolle
Flagge	Ein Klick auf das Bild öffnet die Länderauswahl (siehe unten).
Ländername	Name des Landes in der gewählten Sprache geschrieben (Endonym). Nicht bearbeitbares Feld, auf zwei Zeilen angezeigt.

Rechte Spalte — die Einstellungen der Benutzeroberfläche:

Einstellung	Beschreibung
Sprache	Sprache der Benutzeroberfläche, sofort angewendet. Das Feld bleibt deaktiviert, solange kein Land gewählt ist; es bietet dann die in diesem Land gesprochenen Sprachen an.
Design	Dunkel, Hell oder System.
Schriftgröße	Größe des Textes der Benutzeroberfläche.
Zeilenumbrüche verarbeiten	Interpretiert \n-Sequenzen in Beschreibungen.
Max. zuletzt verwendete Dateien	Anzahl der beibehaltenen Einträge.
Fensterbreite / -höhe	Gespeicherte Abmessungen des Hauptfensters.

Länderauswahl — Ein Klick auf die Flagge öffnet das Fenster „Länderauswahl“, das die 244 Flaggen als Raster mit vertikalem und horizontalem Scrollen darstellt. Es ist ein freies Fenster, das beim Öffnen auf dem Hauptfenster zentriert wird (siehe § 2.6). Ein Feld „Filter:“ ermöglicht es, die Liste nach französischem Namen, englischem Namen oder Ländercode einzugrenzen; ein Löschsymbol (rotes Kreuz) rechts vom Feld leert den Filter mit einem einzigen Klick. Der Ländername erscheint unter jeder Flagge.

Das Raster kann auch über die Tastatur navigiert werden: Die Pfeiltasten (oder der Ziffernblock 8/2/4/6) bewegen den Auswahlrahmen, Page Up/Page Down scrollen um einen Bildschirm, Home/End springen zur ersten und letzten Flagge, Tab und Shift+Tab bewegen sich um eine Flagge vor und zurück, Enter bestätigt das hervorgehobene Land und Esc schließt das Fenster. Einzelheiten finden Sie in Anhang B.

Die Wahl des Landes bestimmt die Liste der angebotenen Sprachen: XIMOD stützt sich auf die Datei CountryLanguages.json, die alle in jedem Land gesprochenen Sprachen auflistet — bis zu 844 für Papua-Neuguinea, daher das Scrollen der Dropdown-Liste.

In Version 2 wurden drei Einstellungen hinzugefügt: „Beizubehaltende Sicherungen“ (Anzahl der früheren Versionen der FOMOD-XML, die bei jedem Speichern unter fomod/backups aufbewahrt werden, 0 = keine, Standard 10); „Wiederherstellungskopie automatisch alle N Minuten“ (eine Wiederherstellungskopie jedes geänderten Projekts wird in diesem Intervall im Konfigurationsordner geschrieben und beim nächsten Start nur nach einem anormalen Beenden angeboten, 0 = aus, Standard 5); und „Die Master eines Plugins als Bedingungen hinzufügen“ (siehe § 8.17, standardmäßig aktiviert).

4.6 Skripte vor und nach dem Speichern

XIMOD Architect kann beim Speichern automatisch ein Skript ausführen: ein Skript vor dem Speichern, das unmittelbar vor dem Schreiben der FOMOD-Dateien gestartet wird, und ein Skript nach dem Speichern, das unmittelbar danach gestartet wird. Sie dienen dazu, die Aufgaben rund um die Mod-Generierung zu automatisieren — zum Beispiel das Bereinigen alter Archive vor dem Speichern, das Hochzählen einer Build-Nummer oder das automatische Verpacken des Mods in ein veröffentlichungsbereites Archiv, sobald das FOMOD geschrieben ist.

Sie werden über Werkzeuge → Skript vor dem Speichern… und Werkzeuge → Skript nach dem Speichern… bearbeitet. Das Fenster zeigt einen Hilfetext, die Liste der verfügbaren Makros, einen Eingabebereich in einer Festbreitenschrift und die Schaltflächen „Speichern“ und „Abbrechen“. Der Inhalt beider Skripte wird in Config.ini (Schlüssel PreSaveScript und PostSaveScript) gespeichert und daher von einer Sitzung zur nächsten beibehalten.

So funktioniert es: Beim Speichern ersetzt XIMOD zunächst die Makros durch ihre Werte, schreibt das Ergebnis in eine temporäre Datei und führt sie dann mit dem Systeminterpreter aus — eine .bat-Befehlsdatei, die unter Windows von cmd gestartet wird, ein .sh-Shellskript, das unter Linux und macOS von sh gestartet wird. Die temporäre Datei wird anschließend gelöscht. Das Skript vor dem Speichern läuft, bevor die XML-Dateien geschrieben werden, das Skript nach dem Speichern unmittelbar danach; ein leer gelassenes Skript wird einfach ignoriert.

Vor der Ausführung werden die folgenden Makros im Skripttext ersetzt:

Makro	Ersetzter Wert
$MODNAME$	Mod-Name.
$MODAUTHOR$	Name des Autors.
$MODVERSION$	Mod-Version.
$MODROOT$	Pfad des Stammverzeichnisses.
$DATE$	Aktuelles Datum (JJJJ-MM-TT).
$TIME$	Aktuelle Uhrzeit (HH:MM:SS).
$RANDOM$	Zufallszahl.

Beispiel — Skript vor dem Speichern (Windows)

Löschen Sie die bereits im Stammverzeichnis des Mods vorhandenen .rar-Archive, um vor dem Speichern mit einem sauberen Ordner zu beginnen:

```
del "$MODROOT$\*.rar" /q
```

Beispiel — Skript nach dem Speichern (Windows)

Sobald das FOMOD geschrieben ist, verpacken Sie automatisch den gesamten Mod-Inhalt in ein .rar-Archiv, das nach dem Mod-Namen und der Version benannt ist, mit WinRAR:

```
d:
cd "$MODROOT$"
"C:\Program Files\WinRAR\rar.exe" a -r -ep1 "$MODNAME$_$MODVERSION$.rar" *
```

Zeile für Zeile: d: wechselt zum Laufwerk D:, cd "$MODROOT$" wechselt zum Stammverzeichnis des Mods, dann erstellt WinRAR das Archiv (-r rekursiv, -ep1 um den übergeordneten Baum nicht wiederzugeben) benannt zum Beispiel Dawnlight_1.2.0.rar.

Entsprechung unter Linux/macOS

Unter Linux und macOS wird das Skript als Shellskript interpretiert. Dasselbe Verpacken nach dem Speichern wird zum Beispiel mit zip geschrieben:

```
cd "$MODROOT$"
zip -r "$MODNAME$_$MODVERSION$.zip" .
```

> Hinweis — Diese Skripte führen echte Systembefehle mit Ihren Benutzerrechten aus; ein Befehl wie del oder rm ist destruktiv. Testen Sie sie an einem Wegwerfprojekt, bevor Sie sie an einem echten Mod verwenden.

4.7 Das Fenster „Über“

Über Hilfe → Über (F1) geöffnet, erinnert dieses Fenster an den Namen der Anwendung, ihre Versionsnummer, eine kurze Beschreibung („Ein plattformübergreifendes Tool zur Erstellung von FOMOD-Installationsprogrammen für Bethesda-Spiel-Mods.“) und den Hinweis „Lizenziert unter MIT“. Eine Schaltfläche „OK“ schließt es.

4.8 Erstellen der FOMOD-Dateien

Beim Speichern übersetzt XIMOD Architect das Projekt in zwei standardisierte XML-Dateien, die in UTF-8 in den Unterordner fomod geschrieben werden.

Vor dem Schreiben wird das Projekt überprüft: Wenn es Anomalien enthält (fehlender Mod-Name, kein Schritt oder keine erforderliche Datei, ein Schritt oder eine Gruppe ohne Namen, eine Gruppe ohne Option), listet ein Fenster sie alle auf und bietet „Trotzdem speichern“ an — das Speichern wird niemals blockiert, was es ermöglicht, ein in Arbeit befindliches Projekt bewusst zu speichern.

Schemavalidierung — Zusätzlich zu dieser Projektprüfung gibt es eine strukturelle Prüfung der generierten ModuleConfig.xml anhand des Schemas ModConfig 5.0, in Rust geschrieben (ohne externe XSD-Engine): Elementhierarchie und Kardinalität, obligatorische Attribute und Aufzählungswerte (Auswahl- und Plugin-Typen, Dateizustände, Operatoren). Jede Anomalie wird durch ihre Zeile und Spalte lokalisiert. Sie ist auf drei Arten zugänglich: über den Befehl Projekt → FOMOD validieren, der einen vollständigen Bericht anzeigt; live im XML-Editor, sobald das Dokument wohlgeformt ist; und integriert in das Bestätigungsfenster beim Speichern.

info.xml enthält die Metadaten unter einem Stamm <fomod>: den Namen (Name), den Autor (Author), die Version (Version), die Website (Website), die Beschreibung (Description), die Kategorie (Groups) und das Spiel (Game). Das Element Game ist eine XIMOD-spezifische Erweiterung, die von Mod-Managern harmlos ignoriert wird und dazu dient, die richtige Kategorienliste beim erneuten Laden wiederherzustellen.

ModuleConfig.xml beschreibt die Installation unter einem Stamm <config>: den Modulnamen, das Kopfbild, die erforderlichen Dateien (requiredInstallFiles), die Schritte (installSteps → group → plugin) und die bedingten Installationen (conditionalFileInstalls). Jedes plugin beschreibt seine Beschreibung, sein Bild, seine Bedingungsflags, seine Dateien und seinen Typ. Abhängigkeiten werden durch einen And- oder Or-Operator kombiniert.

4.9 Übersetzungseditor

Zugänglich über Werkzeuge → Benutzeroberfläche übersetzen…, ermöglicht dieses Werkzeug, die Benutzeroberfläche zu übersetzen, ohne die Dateien von Hand zu bearbeiten, und eine vorhandene Übersetzung zu korrigieren.

Fensterkopf

Feld	Rolle
Flagge	Wahl des Landes, die die Liste der übersetzbaren Sprachen bestimmt.
Länder-Endonymname	Name des Landes in der übersetzten Sprache, angezeigt mit der gewählten Schriftart.
Sprach-Endonymname	Name der Sprache in dieser Sprache selbst (z. B. „Français“).
Autor	Name des Übersetzers.
Angezeigte Sprache	Referenzsprache, die in Spalte 2 angezeigt wird (unter denen, die bereits eine main.ftl-Datei haben).
Zu übersetzende Sprache	Zielsprache, ausgewählt aus den Sprachen des Landes.
Schriftart	Für diese Sprache verwendete Schriftart. Die Schaltfläche „Durchsuchen…“ öffnet eine Dateiauswahl, die auf assets/fonts positioniert ist.
Google Fonts	Öffnet die Website fonts.google.com im Browser.

Wichtige Einschränkung: Die Schriftart muss im Ordner assets/fonts installiert sein. Eine anderswo befindliche Schriftart wird abgelehnt, da das Programm sie über einen relativ zu diesem Ordner angegebenen Pfad lädt und sie beim nächsten Start nicht finden würde. Wenn die Schriftart fehlt, müssen Sie sie daher von Google Fonts herunterladen und dorthin kopieren, bevor Sie sie auswählen.

Ebenso kann nur eine Sprache übersetzt werden, die einen ISO 639-3-Code hat: Dieser Code benennt den Übersetzungsordner und verknüpft die Sprache mit ihrer Schriftart und ihren Referenzdaten. Andernfalls sind die Speicherschaltflächen deaktiviert.

Übersetzungstabelle — drei Spalten:

Spalte 1	Spalte 2	Spalte 3
Schlüssel (nicht bearbeitbar)	Bezeichnung in der angezeigten Sprache	Eingabefeld für die Übersetzung

Token-Schutz: Die Elemente, die nicht übersetzt werden dürfen (Makros wie $MODNAME$, Variablen wie { $num }), erscheinen niemals unverändert in Spalte 2 und werden automatisch bewahrt.

- Token am Anfang oder Ende des Textes: ausgeblendet und dann beim Speichern wieder eingefügt.
- Token in der Mitte des Textes: durch einen kompakten Marker [[1]], [[2]]… ersetzt, der beibehalten und neu positioniert wird.

Der Anwendungsname (Schlüssel app-title) ist bewusst ausgeschlossen und nicht übersetzbar.

Tastaturnavigation — Die Tabelle kann ohne Maus bedient werden (siehe Anhang B): Die Pfeile Auf/Ab und Page Up/Page Down scrollen die Zeilen, Home/End springen zur ersten und letzten, und Enter platziert den Cursor im Eingabefeld der hervorgehobenen Zeile. Während der Bearbeitung wechselt Tab zum nächsten Feld und Shift+Tab zum vorherigen, sodass Übersetzungen nacheinander eingegeben werden können, ohne die Tastatur zu verlassen. Die Anzahl der von Page Up/Page Down bewegten Zeilen passt sich automatisch an die Fensterhöhe an.

Speichern — Die Schaltfläche „Speichern“ schreibt die Datei assets/locales/<Sprache>/main.ftl, der ein Metadatenblock vorangestellt ist, der als Fluent-Kommentare geschrieben ist (die Datei bleibt daher vollkommen gültig):

```
# XIMOD Architect - translation metadata
# @language = fra
# @font = Noto_Sans/static/NotoSans-Regular.ttf
# @langname = Français
# @author = Patrice
```

Der Header beschreibt nur die Sprache: Er enthält keine Länderinformationen, da dieselbe Sprache in mehreren Ländern gesprochen wird (Deutsch in Deutschland sowie in der Schweiz). Das Land wird durch die gewählte Flagge bestimmt, und XIMOD verwaltet die Zuordnung Land → Sprachen aus Countries.json und CountryLanguages.json.

Ausfüllen der Endonymfelder — Wenn das Land (die Flagge) und die Sprache gewählt sind, füllt XIMOD die beiden Felder aus:

- Länder-Endonymname: immer aus Countries.json für das ausgewählte Land aufgelöst; er folgt daher der Flagge (Deutsch zeigt „Deutschland“ für Deutschland, „Schweiz“ für die Schweiz).
- Sprach-Endonymname: aus dem Datei-Header (# @langname) übernommen, wenn er einen hat; andernfalls — eine alte Datei ohne Header oder eine noch nicht übersetzte Sprache — wird er aus Languages.json übernommen.

Die beiden Felder bleiben bearbeitbar: Der Benutzer kann dort Korrekturen vornehmen, die beim Speichern an die Referenzdateien weitergegeben werden.

Das Speichern aktualisiert die Referenzdaten: das Endonym und die Schriftart der Sprache in Languages.json sowie das Länder-Endonym in Countries.json (für das Paar Land/Sprache). Wenn die Sprache dort für dieses Land noch nicht erschien, wird ein Eintrag erstellt.

Da der Header das Land nicht mehr trägt, schreibt XIMOD neben der main.ftl eine kleine tabulatorgetrennte Datei countryEndonyms.tsv, die das Länder-Endonym für die Sprache festhält, eine Zeile pro Land in der Form „Ländercode, Sprachcode, Endonym“. Es ist diese Datei, die es ermöglicht, das Länder-Endonym wiederherzustellen, wenn nur der Sprachordner verfügbar ist.

Senden der Übersetzung — Die Schaltfläche „Senden…“ erstellt ein ZIP-Archiv, das die Übersetzungsdatei (main.ftl in ihrem Sprachordner) und, falls vorhanden, die zugehörige Datei countryEndonyms.tsv enthält; XIMOD öffnet den Archivordner und bereitet dann eine vorausgefüllte E-Mail vor. Die sprachspezifischen Informationen (Sprache, Schriftart, Sprach-Endonym, Autor) reisen im Header der main.ftl; das Länder-Endonym reist in countryEndonyms.tsv. Weder Languages.json noch Countries.json wird angehängt — nur die tatsächlich verwendeten Codes werden übertragen, was die Nutzungsbedingungen der ISO 639-3-Codes respektiert. Der Benutzer hängt die Datei an und sendet sie selbst: Keine Daten verlassen den Rechner ohne eine ausdrückliche Aktion.

> Hinweis — Senden Sie das Paket erst, wenn die Übersetzung vollständig ist: Jeder nicht übersetzte Schlüssel wird mit einem leeren Wert gespeichert, der dann in der Benutzeroberfläche ohne Text angezeigt würde.

4.10 XML-Editor

Zugänglich über Projekt → XML-Editor → info.xml oder ModuleConfig.xml, ermöglicht dieses für fortgeschrittene Benutzer gedachte Werkzeug, das generierte XML anzuzeigen und direkt zu ändern.

Es öffnet sich in einem unabhängigen Fenster, das frei beweglich und in der Größe veränderbar ist, auch auf einem zweiten Bildschirm.

Zwei Modi

- *Schreibgeschützt (Standard): Das XML wird mit Syntaxhervorhebung und hängendem Einzug angezeigt — zu lange Zeilen werden umgebrochen und am Anfang der Zeile ausgerichtet, nach Art von Notepad++. Das Hauptfenster bleibt nutzbar.*
- *Bearbeiten (Schaltfläche „Ändern“): Der Text wird bearbeitbar und die Registerkarten des Hauptfensters werden gesperrt, sodass eine einzige Quelle maßgeblich ist.*

Syntaxhervorhebung: Tags, Attributnamen, Werte in Anführungszeichen, Kommentare, Deklaration und Text werden durch Farben unterschieden, die für das helle oder dunkle Design geeignet sind.

Zeilennummerierung: Eine Leiste zeigt die Nummern; die Fortsetzungszeilen eines Umbruchs bleiben leer.

Live-Validierung: Während der Eingabe wird das XML fortlaufend überprüft. Ein grüner Indikator bestätigt ein wohlgeformtes Dokument; im Fehlerfall gibt eine rote Meldung die Zeile und Spalte an, und das fehlerhafte Token wird unterstrichen.

Änderungen anwenden: Die Schaltfläche „Übernehmen“ analysiert den Text erneut und aktualisiert das Projekt. Im Fehlerfall wird die Änderung abgelehnt und der Text bleibt bearbeitbar. Die Schaltfläche „Abbrechen“ generiert den Text aus dem Projekt neu.

4.11 Schriftartenverwaltung

XIMOD Architect wird mit einem Satz von Noto-Schriftarten ausgeliefert, die alle Schriftsysteme der aufgeführten Sprachen abdecken und im Ordner assets/fonts abgelegt sind.

Diese Schriftarten werden dynamisch geladen: Zu jedem Zeitpunkt sind nur die tatsächlich benötigten installiert, nämlich die Schriftart der Sprache der Benutzeroberfläche und die der Sprachen des ausgewählten Landes. Die Last bleibt gering — eine einzige Schriftart für die meisten Länder.

Dieser Mechanismus gewährleistet die korrekte Anzeige nicht-lateinischer Schriften (Thai, Burmesisch, Amharisch, Hmong usw.), die andernfalls als leere Quadrate erscheinen würden. Er gilt sowohl für die Benutzeroberfläche als auch für das Fenster „Eigenschaften“ und den Übersetzungseditor: Die Schriftarten jeder dort angezeigten Sprache werden geladen, sodass kein Name und keine Bezeichnung als Quadrate erscheint.

XIMOD Architect wird mit 32 Übersetzungen der Benutzeroberfläche ausgeliefert, die jeweils den gesamten Satz von Zeichenketten abdecken, einschließlich nicht-lateinischer Schriften: Japanisch (jpn), vereinfachtes Chinesisch (zho), Koreanisch (kor) und Russisch (rus). Englisch (eng) und Französisch (fra) sind die Referenzübersetzungen, die von Hand gepflegt werden; die anderen stammen aus Beiträgen, die Muttersprachler über den Übersetzungseditor überprüfen können.

Die jeder Sprache zugeordnete Schriftart ist in Languages.json angegeben und kann über den Übersetzungseditor geändert werden.

4.12 Zwei FOMODs zusammenführen

Der Befehl Datei → FOMOD zusammenführen… ermöglicht es, zwei Installationsprogramme zu kombinieren, indem ihr Installationsinhalt zusammengeführt wird. Das aktuell geöffnete Projekt fungiert als Empfänger; es muss daher zuvor erstellt oder geöffnet worden sein, andernfalls bleibt der Befehl deaktiviert.

Ein Klick öffnet eine Dateiauswahl: Wählen Sie die Datei ModuleConfig.xml des zu integrierenden FOMOD (des Gebers). Seine Schritte, erforderlichen Dateien und bedingten Installationen werden dann am Ende derjenigen des Empfängerprojekts hinzugefügt. Die Metadaten des Empfängerprojekts — Name, Autor, Version, Kategorie, Kopfbild — bleiben erhalten: Nur der Installationsinhalt wird importiert.

Nach dem Zusammenführen wird das Projekt als geändert markiert: Denken Sie daran, es zu speichern (Ctrl+S), um das Ergebnis in die FOMOD-Dateien zu schreiben. Das Zusammenführen kann wiederholt werden, um mehrere FOMODs nacheinander zu integrieren.

> Hinweis — Nur die Datei ModuleConfig.xml des Gebers wird gelesen; seine eigenen Metadaten (info.xml) werden nicht importiert. Überprüfen Sie nach dem Zusammenführen, ob Prioritäten und Bedingungen über die kombinierten Schritte hinweg konsistent bleiben.

4.13 Exportieren des Distributionsarchivs

Der Befehl Datei → Distributionsarchiv exportieren… erzeugt in einem einzigen Vorgang das zum Hochladen bereite Archiv (Nexus Mods oder ein beliebiger anderer Mod-Manager). XIMOD schreibt zunächst das FOMOD (fomod/info.xml und fomod/ModuleConfig.xml) in das Stammverzeichnis und komprimiert dann den gesamten Ordner im ZIP-Format. Der Befehl ist deaktiviert, solange kein Stammverzeichnis festgelegt ist.

Eine Auswahl bietet einen Standardnamen der Form „<Name>-<Version>.zip“ an (bereinigt von unzulässigen Zeichen); Sie können ihn vor dem Speichern ändern. Die Statusleiste gibt dann die Anzahl der geschriebenen Dateien und den Pfad des Archivs an.

Archivaufbau — Der Unterordner fomod/ und die Mod-Dateien befinden sich im Stammverzeichnis des Archivs, genau so, wie Mod-Manager es erwarten. Interne Pfade verwenden den Schrägstrich „/“ (plattformübergreifende Kompatibilität).

> Hinweis — Überflüssige Dateien (.git, .DS_Store, Thumbs.db, desktop.ini…) und das Archiv selbst werden automatisch ausgeschlossen. Stellen Sie daher sicher, dass sich alle Mod-Dateien tatsächlich unter dem Stammverzeichnis befinden, bevor Sie exportieren.

4.14 Das Fenster „Eigenschaften“

Über Werkzeuge → Eigenschaften… geöffnet, bietet dieses unabhängige Fenster — beweglich und in der Größe veränderbar, auch auf einem zweiten Bildschirm — einen schreibgeschützten Explorer der eingebetteten Länder-/Sprachdatenbank. Die Datenbank kann hier nicht geändert werden; es handelt sich um ein Nachschlagewerkzeug.

Registerkarte „Länder“: Für ein gewähltes Land zeigt das Fenster seine Flagge, seine französischen und englischen Namen, seine Amtssprachen (mit Endonym, Schriftart und ISO 639-3-/639-1-Codes) und die Liste aller dort gesprochenen Sprachen an.

Registerkarte „Sprachen“: Für eine gewählte Sprache zeigt das Fenster ihre ISO-Codes, ihre Schriftart und führt die umgekehrte Suche nach den Ländern durch, in denen diese Sprache gesprochen wird. Die

Liste „Gesprochen in“ nennt jedes Land in der Sprache, die Sie in den Einstellungen gewählt haben (Flagge → Land → Sprache): sein Endonym in dieser Sprache aus Countries.json, wenn die Sprache dort offiziell ist („Confederazione Svizzera“ für einen italienischsprachigen Benutzer), andernfalls der übersetzte Name aus CountryNames.json („Südafrika“ für einen deutschsprachigen Benutzer) und als letztes Mittel der englische Name.

Tastaturnavigation — Sowohl die Länderliste als auch die Sprachliste können über die Tastatur bedient werden (siehe Anhang B): Pfeile Auf/Ab, Page Up/Page Down (deren Schrittweite sich an die Fensterhöhe anpasst), Home/End und Enter zum Auswählen des hervorgehobenen Elements. Tab und Shift+Tab bewegen die Auswahl innerhalb der Liste, ohne jemals auf der Flagge zu landen. Jedes Feld „Filter:“ hat rechts davon ein Löschsymbol (rotes Kreuz), und das Fenster schließt sich mit Esc.

4.15 Befehlszeilenmodus (CLI)

Mit Argumenten gestartet, öffnet XIMOD Architect kein Fenster: Es führt den angeforderten Befehl aus und beendet sich dann mit einem Exit-Code (0 bei Erfolg, ungleich null bei Fehlschlag oder Validierungsanomalie). Dieser Modus ermöglicht es, das Neuerstellen und Verpacken eines oder mehrerer FOMODs in einem Skript oder einer Continuous-Integration-Pipeline zu automatisieren.

> Hinweis — Unter Windows, da die ausführbare Datei als „windows“-Anwendung kompiliert ist, hängt sie sich an die Konsole an, die sie gestartet hat (über AttachConsole), um dort ihre Meldungen zu schreiben. Starten Sie sie von einer Eingabeaufforderung (cmd oder PowerShell), um die Ausgabe zu sehen; ein Doppelklick auf einen CLI-Befehl würde nichts anzeigen.

Die verfügbaren Befehle sind:

Befehl	Wirkung
ximod-architect validate <root>	Validiert das FOMOD des Verzeichnisses (Projektprüfungen und Konformität mit dem Schema ModConfig 5.0); jede Anomalie wird lokalisiert. Exit-Code ungleich null, wenn eine Anomalie gefunden wird.
ximod-architect build <root>	(Neu-)Schreibt fomod/info.xml und fomod/ModuleConfig.xml aus dem vorhandenen Projekt.
ximod-architect package <root> [-o FILE]	Schreibt das FOMOD und erstellt dann das .zip-Distributionsarchiv. Mit der Option -o können Sie den Ausgabepfad angeben; andernfalls wird das Archiv im aktuellen Verzeichnis erstellt.
ximod-architect help	Zeigt die Hilfe an (Liste der Befehle).
ximod-architect version	Zeigt die Versionsnummer an.

Hier bezeichnet <root> das Stammverzeichnis des Mods (den Ordner, der den Unterordner fomod enthält — oder enthalten wird).

4.16 Nach Updates suchen

Standardmäßig prüft XIMOD Architect einmal täglich beim Start, ob eine neuere Version verfügbar ist. Wenn ja, erscheint ein dezentes Banner unterhalb der Menüleiste mit Links zum Herunterladen (Nexus und GitHub); Sie können auch jederzeit über Hilfe → Nach Updates suchen prüfen. Dies kann im Fenster „Einstellungen“ deaktiviert werden („Beim Start nach Updates suchen“).
Datenschutz: Diese Prüfung ist die einzige Netzwerkanfrage, die die Anwendung stellt. Sie sendet keine persönlichen Daten — nur eine einzige sichere (HTTPS-)Anfrage an die öffentliche GitHub-Releases-Seite, um die neueste Versionsnummer zu lesen. XIMOD lädt oder installiert niemals etwas von selbst; Sie entscheiden immer, ob Sie dem Download-Link folgen.

5. FOMOD-Tag-Referenz

Dieser Abschnitt beschreibt die tatsächlich von XIMOD Architect erzeugten Tags. Die beiden Dateien werden in UTF-8 (mit Byte Order Mark) im Unterordner fomod des Mods geschrieben: info.xml wird in Schritten von vier Leerzeichen eingerückt, ModuleConfig.xml mit einem Tabulator. Beim Lesen wird die Kodierung dank des Byte Order Mark automatisch erkannt, was es ermöglicht, sowohl UTF-8- als auch UTF-16-Dateien (LE oder BE) zu öffnen, die von anderen Werkzeugen erzeugt wurden.

5.1 Die Datei info.xml

Die Metadaten unter einem Stamm <fomod>. Die Elemente werden in der Reihenfolge der untenstehenden Tabelle geschrieben; die mit „wenn nicht leer“ markierten werden weggelassen, wenn das entsprechende Feld leer gelassen wird.

Tag	Inhalt	Geschrieben
`<Name>`	Mod-Name.	Immer
`<Author>`	Autor.	Wenn nicht leer
`<Version>`	Version.	Wenn nicht leer
`<Website>`	URL der Mod-Seite.	Wenn nicht leer
`<Description>`	Beschreibung.	Wenn nicht leer
`<Groups>`	Kategorie des Mods.	Immer
`<Game>`	Bezeichner des ausgewählten Spiels. XIMOD-spezifische Erweiterung (außerhalb der FOMOD-Spezifikation), von Mod-Managern ignoriert.	Wenn nicht leer

Vollständiges Beispiel:

```xml
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>My Awesome Mod</Name>
    <Author>Patrice</Author>
    <Version>1.0.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/12345</Website>
    <Description>A sample mod for the manual.</Description>
    <Groups>Player homes</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

5.2 Die Datei ModuleConfig.xml

Die Installationslogik unter einem Stamm <config>, der das Schema ModConfig5.0.xsd referenziert. Die Blöcke erscheinen in der folgenden Reihenfolge: Modulname, Kopfbild, erforderliche Dateien, Installationsschritte, bedingte Installationen.

Stamm und Kopfzeile

Tag / Attribut	Rolle
`<config>`	Stamm. Trägt die Attribute xmlns:xsi und xsi:noNamespaceSchemaLocation (Schema ModConfig5.0.xsd).
`<moduleName>`	Modulname, oben im Assistenten angezeigt.

Tag / Attribut	Rolle
`<moduleImage path="…"/>`	Kopfbild (leeres Tag, nur geschrieben, wenn ein Bild definiert ist).

Dateien und Ordner

Tag / Attribut	Rolle
`<file source="…" destination="…" priority="…"/>`	Kopie einer Datei. source = Pfad im Archiv (relativ zum Stamm); destination = Ziel unter Data; priority = Installationsreihenfolge.
`<folder source="…" destination="…" priority="…"/>`	Kopie eines gesamten Ordners (gleiche Attribute).
`<requiredInstallFiles>`	Container der systematisch installierten Dateien.
`<files>`	Container der Dateien, die an eine Option oder ein bedingtes Muster gebunden sind.

Schritte und Gruppen

Tag / Attribut	Rolle
`<installSteps order="Explicit">`	Container der Schritte (die definierte Reihenfolge bleibt erhalten).
`<installStep name="…">`	Eine Seite des Assistenten.
`<visible>`	Sichtbarkeitsbedingungen des Schritts (enthält <dependencies>).
`<optionalFileGroups>`	Container der Optionsgruppen des Schritts.
`<group name="…" type="…">`	Eine Gruppe; type = Auswahlregel (siehe unten).
`<plugins order="Explicit">`	Container der Optionen der Gruppe.

Werte des Attributs type einer <group>:

type	Einschränkung
Genau eines (erforderlich) — SelectExactlyOne	Genau eine Option.
Höchstens eines — SelectAtMostOne	Höchstens eine Option.
Beliebig viele — SelectAny	Beliebig viele.
Alle (keine Auswahl) — SelectAll	Alle, erzwungen.
Mindestens eines — SelectAtLeastOne	Mindestens eine.

Optionen (Plugins)

Tag / Attribut	Rolle
`<plugin name="…">`	Eine dem Benutzer angebotene Option.
`<description>`	Beschreibungstext der Option (immer geschrieben).
`<image path="…"/>`	Bild, das die Option illustriert (wenn definiert).

Tag / Attribut	Rolle
`<conditionFlags>`	Container der Flags, die gesetzt werden, wenn die Option gewählt wird.
`<flag name="…">value</flag>`	Ein Flag: Name im Attribut, Wert im Inhalt.
`<typeDescriptor>`	Beschreibt den Typ der Option (einfach oder bedingt).
`<type name="…"/>`	Einfacher Typ (wenn kein Abhängigkeitsmuster definiert ist).
`<dependencyType>`	Dynamischer Typ (wenn Muster existieren).
`<defaultType name="…"/>`	Standardtyp, angewendet, wenn kein Muster zutrifft.

Werte eines <type> / <defaultType>:

Typ	Wirkung im Assistenten
Optional — Optional	Option nicht angekreuzt, frei aktivierbar.
Erforderlich — Required	Obligatorische Option, kann nicht abgewählt werden.
Empfohlen — Recommended	Standardmäßig angekreuzte Option, kann abgewählt werden.
Nicht verwendbar — NotUsable	Deaktivierte Option (kann nicht ausgewählt werden).
Könnte verwendbar sein — CouldBeUsable	Verwendbar, aber als potenziell riskant gekennzeichnet.

Abhängigkeiten und Bedingungen

Tag / Attribut	Rolle
`<dependencies operator="…">`	Gruppe von Bedingungen; operator = And oder Or.
`<fileDependency file="…" state="…"/>`	Bedingung für eine Datei; state = Active, Inactive oder Missing.
`<flagDependency flag="…" value="…"/>`	Bedingung für ein Flag und seinen erwarteten Wert.
`<conditionalFileInstalls>`	Abschnitt der bedingten Installationen.
`<patterns> / <pattern>`	Liste von Mustern; jedes Muster verknüpft <dependencies> mit <files> (oder, innerhalb eines typeDescriptor, mit einem <type>).

6. Die Datei ModuleConfig.xml im Detail

Kapitel 5 hat die Tags als Referenztabellen vorgestellt. Dieses Kapitel geht sie einzeln in verständlicher Sprache durch, um zu erklären, was jedes tut, welche Optionen es akzeptiert und vor allem, welche konkrete Wirkung diese Optionen auf den Installationsassistenten haben, den der Benutzer des Mods zu sehen bekommt. Keine Vorkenntnisse über XML sind erforderlich: XIMOD Architect schreibt diese Datei für Sie; sie zu verstehen hilft nur dabei, zuverlässigere Installationsprogramme zu entwerfen.

Ein paar Worte zum Vokabular. Eine XML-Datei besteht aus „Tags“, die in spitze Klammern eingeschlossen sind. Ein öffnendes Tag <step> gehört mit einem schließenden Tag </step> zusammen; was auch immer zwischen beiden geschrieben steht, gehört zu diesem Tag. Ein Tag kann „Attribute“ tragen, das heißt Einstellungen, die in der Form name="wert" geschrieben werden. So ist <group name="Resolution" type="SelectExactlyOne"> ein „group“-Tag mit zwei Attributen. Schließlich schließt sich ein Tag ohne Inhalt selbst, wie <file … /> (beachten Sie den abschließenden /).

6.1 Wie der Installer die Datei liest

Wenn der Benutzer Ihren Mod installiert, liest sein Manager (Vortex, MO2, NMM) ModuleConfig.xml von oben nach unten und baut einen Assistenten: eine Reihe von Seiten (die Schritte), die jeweils Gruppen von Optionen zum Ankreuzen anbieten. Am Ende sammelt der Manager alle Dateien, die den gewählten Optionen entsprechen, und kopiert sie in das Spiel. Drei Ideen, die man im Kopf behalten sollte:

- Die Reihenfolge ist wichtig: Die Schritte werden in der Reihenfolge angezeigt, in der sie in der Datei erscheinen;
- Das Ankreuzen installiert nichts sofort: Das Ankreuzen einer Option zeichnet nur eine Auswahl auf; das eigentliche Kopieren der Dateien geschieht ganz am Ende;
- Bedingungen können alles modulieren: einen Schritt ausblenden, den Zustand einer Option ändern oder die Installation zusätzlicher Dateien auslösen — dies ist die Rolle von Flags und Abhängigkeiten (§ 6.6).

6.2 Die Wurzel und der Header

<config> — die Wurzel. Die gesamte Konfiguration passt in ein einziges Tag <config>…</config>. Es trägt zwei technische Attribute (xmlns:xsi und xsi:noNamespaceSchemaLocation), die das Validierungsschema (ModConfig 5.0) bezeichnen. Sie müssen sie niemals anfassen: XIMOD schreibt sie immer identisch.

<moduleName> — der Mod-Name, oben im Assistenten angezeigt. Es ist der auf der Registerkarte „Mod-Info“ eingegebene.

<moduleImage path="…"/> — das Bannerbild, oben im Assistenten angezeigt. Das Attribut path gibt seinen Speicherort relativ zum Stamm des Mods an (z. B. fomod\header.png). Das Tag ist leer (es schließt sich selbst). Ohne Bild ist es einfach nicht vorhanden.

6.3 Dateien installieren

Zwei Tags beschreiben, was in das Spiel kopiert wird: <file> kopiert eine einzelne Datei, <folder> kopiert einen gesamten Ordner mit seinem gesamten Inhalt. Beide akzeptieren dieselben drei Attribute:

Attribut	Rolle	Beispiel

Attribut	Rolle	Beispiel
`source`	Pfad des Elements im Archiv Ihres Mods, relativ zum Stamm.	`tex\2k`
`destination`	Wohin das Element gelangen muss, relativ zum Data-Ordner des Spiels.	`textures`
`priority`	Installationsreihenfolge im Konfliktfall (siehe unten).	`0`

Das Attribut priority und seine Auswirkung. Es ist eine Zahl, die die Kopierreihenfolge festlegt, wenn zwei Optionen eine Datei an denselben Ort installieren. Die Regel: Niedrige Prioritäten werden zuerst installiert, hohe danach — und da die zuletzt geschriebene gewinnt, „überschreibt“ eine höhere Priorität eine niedrigere. Beispiel: Wenn eine Basisoption eine Textur mit priority="0" installiert und eine Option „hohe Auflösung“ dieselbe Textur mit priority="1" installiert, bleibt die hochauflösende Version erhalten, unabhängig von der Anzeigereihenfolge der Optionen. Lassen Sie überall 0, solange Sie keinen bewussten Konflikt zu entscheiden haben.

> Hinweis — Das Ziel ist standardmäßig leer, was das Element im Stammverzeichnis des Data-Ordners des Spiels installiert. Klicken Sie auf die Zelle „Ziel“, um einen expliziten Pfad (relativ zu Data) einzugeben, wann immer das Element woanders hin muss. Ein leeres Ziel ist die richtige Wahl, wenn der Quellordner die Struktur des Spiels bereits spiegelt.

Diese Tags erscheinen an drei Stellen: in <requiredInstallFiles> (immer installierte Dateien), in den <files> einer Option (installiert, wenn die Option gewählt wird) und in den <files> eines bedingten Musters (installiert, wenn die Bedingungen erfüllt sind).

6.4 Schritte, Gruppen und Optionen

Dies ist das Herzstück des Assistenten. Die Hierarchie ist einfach: Ein Schritt enthält Gruppen, eine Gruppe enthält Optionen.

<installSteps order="Explicit"> fasst alle Schritte zusammen; order="Explicit" bedeutet „respektiere die Reihenfolge, die ich geschrieben habe“. Jede Seite des Assistenten ist ein <installStep name="…">, dessen Attribut name der oben auf der Seite angezeigte Titel ist.

<visible> — den Schritt bedingt anzeigen. Ein Schritt kann nur angezeigt werden, wenn bestimmte Bedingungen wahr sind; Sie platzieren sie in einem Tag <visible> (siehe § 6.6). Ohne <visible> wird der Schritt immer angezeigt. Beispiel: einen Schritt „Erweiterte Einstellungen“ nur anzeigen, wenn der Benutzer zuvor „Expert-Installation“ angekreuzt hat.

<optionalFileGroups> enthält die Gruppen des Schritts. Jede <group name="…" type="…"> hat einen Namen (Abschnittstitel) und vor allem einen type, der die Auswahlregel festlegt — das heißt, was der Benutzer ankreuzen darf:

type	Was der Benutzer sieht und tun kann
Genau eines (erforderlich) — SelectExactlyOne	Optionsfelder: Sie müssen eine Option wählen, und nur eine. Weder null noch zwei. Ideal für exklusive Varianten (eine einzige Texturauflösung).
Höchstens eines — SelectAtMostOne	Wie oben, aber sie können auch nichts wählen: eine „Option oder nichts“.
Beliebig viele — SelectAny	Unabhängige Kontrollkästchen: Sie kreuzen an, was sie wollen, von keiner bis zu allen. Der häufigste Fall, für stapelbare Module.

type	Was der Benutzer sieht und tun kann
Mindestens eines — SelectAtLeastOne	Kontrollkästchen, aber sie müssen mindestens eines ankreuzen, um fortzufahren. Verhindert, dass man mit leeren Händen weitergeht.
Alle (keine Auswahl) — SelectAll	Alle Optionen werden automatisch angekreuzt und gesperrt: Der Benutzer kann sie nicht abwählen. Wird verwendet, um obligatorische Dateien zu präsentieren und sie dabei zu beschreiben.

In einer Gruppe enthält <plugins order="Explicit"> die Optionen, von denen jede ein <plugin name="…"> ist (das Wort „plugin“ bedeutet hier eine Option des Assistenten, nicht eine .esp-Datei). Sein name ist die Bezeichnung, die der Benutzer ankreuzt. Eine Option besteht aus:

- <description>: der Text, der rechts angezeigt wird, wenn die Option ausgewählt ist — beschreiben Sie dort klar ihre Wirkung;
- <image path="…"/>: eine Illustration, die mit der Beschreibung angezeigt wird (optional);
- <conditionFlags>: die Flags, die gesetzt werden, wenn die Option gewählt wird (§ 6.6);
- <files>: die Dateien, die installiert werden, wenn die Option gewählt wird;
- <typeDescriptor>: der Standardzustand der Option (§ 6.5).

6.5 Der Typ einer Option und ihr Verhalten

Jede Option hat einen „Typ“, der ihren Standardzustand im Assistenten bestimmt: angekreuzt oder nicht, bearbeitbar oder nicht. Er wird durch <typeDescriptor> beschrieben. Im einfachen Fall enthält er ein einziges <type name="…"/>. Die fünf Werte und ihre Wirkung:

type	Wirkung im Assistenten
Optional — Optional	Standardmäßig nicht angekreuzt, frei an- und abwählbar. Der häufige Fall.
Empfohlen — Recommended	Standardmäßig angekreuzt, aber der Benutzer kann es abwählen. Für eine empfohlene Wahl.
Erforderlich — Required	Angekreuzt und gesperrt: kann nicht abgewählt werden. Für eine wesentliche Komponente.
Nicht verwendbar — NotUsable	Nicht angekreuzt und gesperrt: kann nicht angekreuzt werden. Zeigt eine im aktuellen Kontext nicht verfügbare Option an.
Könnte verwendbar sein — CouldBeUsable	Ankreuzbar, aber als potenziell problematisch gekennzeichnet (oft mit einer Warnung).

Dynamische Typen: <dependencyType>. Der Typ einer Option kann sich auch je nach Bedingungen ändern. Anstelle eines festen <type> schreiben Sie ein <dependencyType>, das ein <defaultType name="…"/> (den Standardtyp, wenn keine Bedingung erfüllt ist) und eine Liste von <pattern> enthält: Jedes Muster verknüpft Bedingungen mit einem <type>, der anzuwenden ist, wenn sie wahr sind.

Beispiel: Eine Option „ENB Ultra-Preset“ kann standardmäßig Optional sein, Recommended werden, wenn der Benutzer dynamische Kerzen aktiviert hat, und NotUsable werden, wenn eine erforderliche Datei auf der Festplatte fehlt. Es ist dieser Mechanismus, der ein Installationsprogramm „intelligent“ macht. Ein vollständiges Beispiel erscheint in Anhang D. Abschnitt 7.5 nutzt dies, um ein „Preset“ in Standardwerte zu verwandeln, die der Benutzer weiterhin anpassen kann.

6.6 Flags, Abhängigkeiten und bedingte Installationen

Diese drei Begriffe bilden das FOMOD-Bedingungssystem — das, was es einem Installationsprogramm ermöglicht, auf die Auswahl des Benutzers zu reagieren.

Flags — <conditionFlags> und <flag>. Ein Flag ist eine kleine Variable, die Sie setzen, wenn eine Option gewählt wird. Das Tag <conditionFlags> einer Option enthält ein oder mehrere <flag name="…">wert</flag>: name ist der Name des Flags, und der Text zwischen den Tags ist sein Wert. So setzt <flag name="res">4K</flag> das Flag „res“ auf „4K“, sobald die Option angekreuzt wird. Ein Flag hat für sich allein keine sichtbare Wirkung: Es dient als Gedächtnis, das später an anderer Stelle als Bedingung wiederverwendet wird.

Abhängigkeiten — <dependencies operator="…">. Eine Abhängigkeit ist eine zu prüfende Bedingung. Sie werden in <dependencies> gruppiert, dessen Attribut operator angibt, wie sie zu kombinieren sind: And (alle Bedingungen müssen wahr sein) oder Or (eine einzige genügt). Es gibt zwei Arten von Bedingungen:

Bedingung	Was sie prüft
`<flagDependency flag="…" value="…"/>`	Wahr, wenn das benannte Flag genau den angegebenen Wert hat. So reagieren Sie auf zuvor getroffene Auswahlentscheidungen (z. B.: das Flag „res“ ist gleich „4K“).
`<fileDependency file="…" state="…"/>`	Wahr je nach Zustand einer Datei, die im Spiel des Benutzers bereits vorhanden ist oder nicht.

Das Attribut state einer Dateiabhängigkeit nimmt drei Werte an:

state	Bedeutung
Active	Die Datei ist im Spiel vorhanden und aktiviert.
Inactive	Die Datei ist vorhanden, aber deaktiviert.
Missing	Die Datei ist abwesend.

Diese Abhängigkeiten dienen drei Zwecken: der Sichtbarkeit eines Schritts (in <visible>), dem dynamischen Typ einer Option (in einem <pattern> von <dependencyType>) und den bedingten Installationen weiter unten.

Bedingte Installationen — <conditionalFileInstalls>. Dieser Abschnitt, der nach den Schritten platziert ist, installiert Dateien nur dann, wenn Bedingungen erfüllt sind, ohne dass der Benutzer direkt etwas ankreuzen muss. Er enthält <pattern>-Elemente, von denen jedes <dependencies> mit einer <files> verknüpft. Wenn die Bedingungen des Musters am Ende des Assistenten wahr sind, werden seine Dateien installiert.

Unterschied zur Schrittsichtbarkeit: <visible> entscheidet, ob eine Seite angezeigt wird, während <conditionalFileInstalls> entscheidet, ob Dateien installiert werden, hinter den Kulissen. Typischer Fall: automatisch einen Kompatibilitätspatch installieren, nur wenn der Benutzer zwei bestimmte Optionen angekreuzt hat (deren Flags dann beide gesetzt sind).

> Hinweis — Ein Flag muss zuerst von einer Option gesetzt werden (über <conditionFlags>), bevor es von einer Abhängigkeit geprüft werden kann. Definieren Sie also Ihre Flags in den Schritten und prüfen Sie sie dann in den bedingten Mustern.
>
> Hinweis — Entwurfsregel: Wenn eine Option nur für eine Entscheidung steht — ein Preset, ein Profil, ein „Modul“-Schalter —, lassen Sie sie Flags setzen und sonst nichts, und überlassen Sie das Kopieren bedingten Installationen, die diese Flags prüfen. Ein Flag pro Funktion ist dann die einzige maßgebliche Quelle: Wie auch immer die Auswahlen kombiniert werden, ein Preset und eine von Hand gewählte Option können niemals dieselben Dateien zweimal installieren. Abschnitt 7.5 wendet diese Regel an.

7. Beispiele für ModuleConfig.xml

Die folgenden Beispiele veranschaulichen die häufigsten Fälle. Sie entsprechen dem tatsächlich von XIMOD Architect erzeugten XML (Einrückung zur besseren Lesbarkeit vereinfacht).

7.1 Minimalbeispiel — erforderliche Dateien

Ein Mod ohne Auswahlmöglichkeiten: Alle Dateien werden direkt installiert.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>My Simple Mod</moduleName>
    <requiredInstallFiles>
        <file source="MyMod.esp" destination="MyMod.esp" priority="0"/>
        <folder source="textures" destination="textures" priority="0"/>
    </requiredInstallFiles>
</config>
```

7.2 Exklusive Auswahl — SelectExactlyOne

Ein Schritt, der die Wahl einer einzigen Texturauflösung anbietet; die 2K-Option ist standardmäßig empfohlen.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>HD Textures</moduleName>
    <moduleImage path="fomod\header.png"/>
    <installSteps order="Explicit">
        <installStep name="Texture resolution">
            <optionalFileGroups>
                <group name="Resolution" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="2K">
                            <description>2K textures (balanced).</description>
                            <files>
                                <folder source="2k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Recommended"/>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="4K">
                            <description>4K textures (high quality).</description>
                            <files>
                                <folder source="4k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
</config>
```

7.3 Bedingte Installation — Flags

Eine Option setzt ein Flag, wenn sie angekreuzt wird; eine Datei wird nur installiert, wenn dieses Flag aktiv ist.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Conditional patch</moduleName>
    <installSteps order="Explicit">
        <installStep name="Options">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enables module A.</description>
                            <conditionFlags>
                                <flag name="ModuleA">On</flag>
                            </conditionFlags>
                            <files>
                                <folder source="A" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
    <conditionalFileInstalls>
        <patterns>
            <pattern>
                <dependencies operator="And">
                    <flagDependency flag="ModuleA" value="On"/>
                </dependencies>
                <files>
                    <file source="patchA.esp" destination="patchA.esp" priority="0"/>
                </files>
            </pattern>
        </patterns>
    </conditionalFileInstalls>
</config>
```

7.4 Eine Seite, mehrere Schritte — SelectAny + Flags

Ein einziger Mehrfachauswahlschritt kann für jede vom Spieler angekreuzte Option einen anderen Schritt öffnen: Schritte sind nicht an Optionen gebunden. Jede Option setzt ein Bedingungsflag, und jeder Zielschritt trägt eine Sichtbarkeitsbedingung für dieses Flag. Das Ankreuzen von A und B zeigt dann die Schritte A und B an; das Ankreuzen von A, B und C zeigt drei Schritte an.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Modular Add-ons</moduleName>
    <installSteps order="Explicit">
        <!-- Step 1: the choices. Each ticked option sets its own flag. -->
        <installStep name="Choose modules">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enable module A.</description>
                            <conditionFlags>
                                <flag name="want_A">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="Module B">
                            <description>Enable module B.</description>
                            <conditionFlags>
                                <flag name="want_B">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- Modules C and D follow the same pattern (want_C, want_D). -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step A: shown only when want_A is on. -->
        <installStep name="Module A options">
            <visible>
                <flagDependency flag="want_A" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module A" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="A - extra textures">
                            <description>Optional high-res textures for A.</description>
                            <files>
                                <folder source="a_textures" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step B: shown only when want_B is on. -->
        <installStep name="Module B options">
            <visible>
                <flagDependency flag="want_B" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module B" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="B - extra meshes">
                            <description>Optional meshes for B.</description>
                            <files>
                                <folder source="b_meshes" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Steps for C and D have exactly the same shape, each guarded by its
             own <visible><flagDependency flag="want_C | want_D" value="on"/>. -->
    </installSteps></config>
```

7.5 Preset mit anpassbaren Standardwerten

Ein „Preset“ ist eine Option, die für mehrere Entscheidungen auf einmal steht. Benutzer wollen beides: das Preset wählen und fertig sein, oder das Preset wählen und trotzdem eine oder zwei Funktionen anpassen. Der Schlüssel liegt darin, das Entscheiden vom Installieren zu trennen. Das Preset entscheidet — es setzt Flags. Auf den Funktionsseiten kann der Benutzer diese Entscheidungen revidieren — sie setzen dieselben Flags erneut. Die bedingten Installationen übernehmen das Kopieren, ausgehend vom endgültigen Wert jedes Flags.

Schritt 1 bietet das Preset an und daneben eine Gruppe „Anpassen“, deren Optionen lediglich die Seiten der Funktionen öffnen, die sich der Benutzer ansehen möchte. Jede Funktionsseite ist sichtbar, wenn ihre Option „Anpassen“ angekreuzt ist (§ 7.4). Auf dieser Seite haben die Optionen einen dynamischen Typ (§ 6.5): Die Variante, die das Preset installiert hätte, wird Recommended — also vorab angekreuzt —, wenn das Preset-Flag gesetzt ist, andernfalls wird „Keine“ empfohlen. Da die Seite nach dem Preset kommt, ersetzt der Wert, den sie in das Funktions-Flag schreibt, den des Presets: Der Benutzer kann die Variante des Presets beibehalten, zu einer anderen wechseln oder die Funktion ganz weglassen.

Weder das Preset noch die Optionen der Seiten enthalten `<files>`. Das gesamte Kopieren geschieht in `<conditionalFileInstalls>`, ein Muster pro Flag-Wert. Genau das macht doppelte Installationen unmöglich: Wie auch immer die Auswahlen kombiniert werden, jedes Funktions-Flag endet mit genau einem Wert und damit mit genau einem Satz von Dateien.

> Hinweis — Mod-Manager werten die Flags Seite für Seite aus, sodass eine Zuweisung auf einer späteren Seite die des Presets ersetzt. Platzieren Sie das Preset auf der ersten Seite und geben Sie jeder Funktionsseite eine Option „Keine“, die das Flag ausdrücklich zurücksetzt: Ein Benutzer, der lediglich alles abwählt, würde sonst den Wert des Presets behalten.

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Preset mit anpassbaren Standardwerten</moduleName>
    <installSteps order="Explicit">
        <!-- Schritt 1: Das Preset setzt die Flags aller Funktionen, die es enthält, und sonst nichts. --
>
        <installStep name="Installationsmodus">
            <optionalFileGroups>
                <group name="Preset" type="SelectAtMostOne">
                    <plugins order="Explicit">
                        <plugin name="Preset A (empfohlene Konfiguration)">
                            <description>Installiert Funktion A in Version 2 und Funktion B in Version
1.</description>
                            <conditionFlags>
                                <flag name="preset">on</flag>
                                <flag name="feat_A">v2</flag>
                                <flag name="feat_B">v1</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
                <group name="Anpassen" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Funktion A anpassen">
                            <description>Öffnet die Seite von Funktion A.</description>
                            <conditionFlags><flag name="cust_A">on</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- „Funktion B anpassen“, „Funktion C anpassen“... setzen cust_B, cust_C. -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Schritt 2: Funktion A, nur angezeigt, wenn der Benutzer sie anpassen wollte. -->
        <installStep name="Funktion A">
            <visible><flagDependency flag="cust_A" value="on"/></visible>
```

```
            <optionalFileGroups>
                <group name="Funktion A" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="Keine">
                            <description>Funktion A nicht installieren.</description>
                            <conditionFlags><flag name="feat_A">none</flag></conditionFlags>
                            <typeDescriptor>
                                <dependencyType>
                                    <defaultType name="Recommended"/>
                                    <patterns>
                                        <pattern>
                                            <dependencies operator="And">
                                                <flagDependency flag="preset" value="on"/>
                                            </dependencies>
                                            <type name="Optional"/>
                                        </pattern>
                                    </patterns>
                                </dependencyType>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="Version 1">
                            <description>Funktion A, Version 1.</description>
                            <conditionFlags><flag name="feat_A">v1</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="Version 2">
                            <description>Funktion A, Version 2 (Wahl des Presets).</description>
                            <conditionFlags><flag name="feat_A">v2</flag></conditionFlags>
                            <typeDescriptor>
                                <dependencyType>
                                    <defaultType name="Optional"/>
                                    <patterns>
                                        <pattern>
                                            <dependencies operator="And">
                                                <flagDependency flag="preset" value="on"/>
                                            </dependencies>
                                            <type name="Recommended"/>
                                        </pattern>
                                    </patterns>
                                </dependencyType>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Die Seiten der Funktionen B und C haben genau dieselbe Form. -->
    </installSteps>
    <!-- Dateien werden nur hier kopiert, ausgehend vom endgültigen Wert jedes Funktions-Flags. -->
    <conditionalFileInstalls>
        <patterns>
            <pattern>
                <dependencies operator="And"><flagDependency flag="feat_A" value="v1"/></dependencies>
                <files><folder source="FeatureA\v1" destination="" priority="0"/></files>
            </pattern>
            <pattern>
                <dependencies operator="And"><flagDependency flag="feat_A" value="v2"/></dependencies>
                <files><folder source="FeatureA\v2" destination="" priority="0"/></files>
            </pattern>
            <pattern>
                <dependencies operator="And"><flagDependency flag="feat_B" value="v1"/></dependencies>
                <files><folder source="FeatureB\v1" destination="" priority="0"/></files>
            </pattern>
        </patterns>
    </conditionalFileInstalls>
</config>
```

Mit Projekt → Installationsprogramm-Vorschau… können Sie jede Kombination durchspielen — Preset allein, Preset plus „Funktion A anpassen“, „Funktion A anpassen“ allein — und die endgültige Dateiliste vor der Veröffentlichung prüfen (§ 4.11, § 8.19). Projekt → FOMOD validieren meldet Zielkonflikte und erkennt dieses Muster aus Preset und Anpassung (§ 8.1).

7.6 Einleitungs- und Abschlussseiten

Der Assistent wird vom Mod-Manager gezeichnet, nicht vom Installationsprogramm. Ein Autor bestimmt die Namen, Beschreibungen und Bilder der Schritte, Gruppen und Optionen — und sonst nichts: keine Schriftarten, keine Farben, kein Layout, kein freier Textblock. Innerhalb dieser Grenzen lassen sich eine Willkommensseite und eine Abschlussseite leicht erstellen.

Willkommensseite — Bauen Sie den ersten Schritt aus einer einzigen Gruppe vom Typ SelectAll auf, die eine Option vom Typ Required enthält. Deren Beschreibung trägt den Willkommenstext, den der Manager im Beschreibungsbereich anzeigt; ihr Bild trägt die Gestaltung — ein Banner, ein Logo oder Text, der als Bild in einer beliebigen Schriftart gerendert ist. Geben Sie ihr keine Dateien oder aber die Kerndateien des Mods, wenn diese dort aufgeführt werden sollen.

Abschlussseite — dieselbe Konstruktion als letzter Schritt: eine Option vom Typ Required mit einer Abschiedsnachricht und, falls sinnvoll, den nächsten Schritten (Ladereihenfolge, zu installierende Patches). Ein Installationsprogramm kann keine Zusammenfassung der Auswahlen des Benutzers anzeigen: Das Format gibt einer Seite keinen Zugriff auf die auf den anderen Seiten getroffenen Entscheidungen. Der Manager zeigt seine eigene Liste der Dateien an, die gleich installiert werden, und die Installationsprogramm-Vorschau (§ 4.11) liefert Ihnen als Autor diese Zusammenfassung für jedes Szenario.

```
<installStep name="Willkommen">
    <optionalFileGroups>
        <group name="Über dieses Installationsprogramm" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="Willkommen bei Dawnlight">
                    <description>Danke, dass Sie Dawnlight heruntergeladen haben. Auf den nächsten Seiten
wählen Sie das Beleuchtungs-Preset, die optionalen Module und die Kompatibilitätspatches. Bis zur letzten
Seite wird nichts installiert.</description>
                    <image path="fomod\welcome.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
<!-- ... die eigentlichen Schritte ... -->
<installStep name="Fertig">
    <optionalFileGroups>
        <group name="Bevor Sie spielen" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="Vielen Dank!">
                    <description>Platzieren Sie Dawnlight.esp nach Ihren Wetter-Mods. Melden Sie Probleme
bitte auf der Nexus-Seite. Viel Spaß!</description>
                    <image path="fomod\thanks.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
```

Speichern Sie den Willkommensschritt als Vorlage (Projekt → Vorlagen…, § 8.3), um ihn in jedem Mod wiederzuverwenden.

8. Version 2 — neue Funktionen

Die folgenden Funktionen wurden mit Version 2.0.0 eingeführt. Sie werden in der Reihenfolge ihrer Hinzufügung vorgestellt; die nummerierten Abschnitte der Kapitel 2 bis 7 spiegeln sie bereits wider. Die Abschnitte 8.26 bis 8.29 kamen mit Version 2.1.0 hinzu.

8.1 Erkennung von Zielkonflikten

Wenn Sie ein Projekt validieren (Projekt → FOMOD validieren), meldet XIMOD nun auch Optionen, die unterschiedliche Dateien an denselben Zielpfad installieren, was sich gegenseitig überschreiben würde. Kollisionen, die gemeinsam auftreten können, werden als sicher gekennzeichnet; solche, die nur bei sich gegenseitig ausschließenden Optionen auftreten, werden als möglich gekennzeichnet.
Ein Konflikt wird nur dann als sicher gemeldet, wenn die beiden Optionen tatsächlich gemeinsam ausgewählt werden können. Neben den exklusiven Gruppen (SelectExactlyOne / SelectAtMostOne) verfolgt XIMOD nun die Flags: Wenn eine Option ein Flag auf einen Wert setzt und die andere Option auf einer Seite liegt, die nur für einen anderen Wert dieses Flags angezeigt wird — und nichts, was neben der ersten Option auswählbar ist, diesen anderen Wert setzen kann —, können die beiden nicht nebeneinander bestehen und der Konflikt wird nur als potenziell aufgeführt. Typischer Fall: eine „Preset“-Option und die entsprechende Option einer „Selbst wählen“-Seite.

8.2 7z-Export

Der Befehl „Exportieren“ bietet nun zusätzlich zu ZIP auch 7-Zip (.7z) an. Wählen Sie das Format im Speicherdialog; 7z erzeugt in der Regel kleinere Downloads.

8.3 Wiederverwendbare Vorlagen

Projekt → Vorlagen… ermöglicht es, den ausgewählten Schritt als wiederverwendbare Vorlage zu speichern und gespeicherte Vorlagen (Schritt, Gruppe oder Option) in jedes Projekt einzufügen. Vorlagen werden im Konfigurationsordner der Anwendung gespeichert.

8.4 Neu aus Ordner

Datei → Neu aus Ordner… durchsucht einen Ordner und schlägt ein Grundgerüst vor, bei dem jeder unmittelbare Unterordner zu einer Option wird; ein offensichtliches Kopfbild im Stammverzeichnis wird automatisch erkannt.

8.5 Dateizuweisung per Drag-and-Drop

Wenn im Baum eine Option ausgewählt ist, ziehen Sie Dateien oder Ordner aus Ihrem Dateimanager (Windows-Explorer, Finder, Ihr Linux-Dateimanager) auf das XIMOD-Fenster: Sie werden der Quelltabelle der Option hinzugefügt, genau wie mit Datei hinzufügen / Ordner hinzufügen, und es können mehrere Elemente auf einmal abgelegt werden. Nur Pfade unterhalb des Stammordners des Projekts werden akzeptiert; alle anderen werden ignoriert. Bei einem großen Installationsprogramm ist dies der schnellste Weg, Quellen zuzuweisen: Öffnen Sie den Funktionsordner einmal im Explorer und legen Sie jeden Unterordner auf seiner Option ab.

8.6 FOMOD-Vergleich

Projekt → Vergleichen mit… vergleicht das aktuelle Projekt mit einem anderen FOMOD und listet auf, was hinzugefügt, entfernt oder geändert wurde (Schritte, Optionen, Dateien und Metadaten) — praktisch zum Schreiben eines Änderungsprotokolls.

8.7 Visueller Bedingungseditor

Projekt → Bedingungseditor… zeigt für jedes Bedingungsflag an, welche Optionen es setzen und welche Schrittsichtbarkeitsregeln, bedingten Installationen und Abhängigkeitsmuster es verwenden. Flags, die gesetzt, aber nie verwendet (oder verwendet, aber nie gesetzt) werden, werden hervorgehoben.

8.8 Bildprüfungen und -optimierung

Die Validierung warnt nun vor Kopf-/Optionsbildern, die zu groß sind, in einem nicht unterstützten Format vorliegen oder nicht lesbar sind. Auf der Registerkarte „Mod-Info“ verkleinert „Bild optimieren“ das Kopfbild auf sinnvolle Grenzen.

8.9 Ergänzungen der Befehlszeile

Die Befehlszeile erhält: inspect <plugin.esp> (gibt die Master, den Autor und das ESL-Flag eines Plugins aus); batch <manifest.json> (validiert, erstellt und verpackt mehrere FOMODs, für Continuous Integration); und ba2 <dir> <out.ba2> [game], um einen Ordner in ein allgemeines BA2-Archiv zu packen. Das BA2-Packen ist experimentell — überprüfen Sie das Archiv im Spiel, bevor Sie es ausliefern.

8.10 Export und Validierung im Hintergrund

Das Exportieren des Distributionsarchivs friert das Fenster nicht mehr ein: Sobald das FOMOD geschrieben ist, läuft die Komprimierung im Hintergrund und ein Fortschrittsfenster zeigt die Anzahl der verarbeiteten Dateien und die gerade hinzugefügte Datei an. Eine Schaltfläche „Abbrechen“ stoppt den Export und entfernt das unvollständige Archiv. Ebenso laufen die Festplattenprüfungen der Validierung (fehlende Dateien, Zielkonflikte, Bilder) im Hintergrund; das Berichtsfenster zeigt „Dateien auf der Festplatte werden überprüft…“ an, bis sie abgeschlossen sind, und warnt Sie, wenn sich das Projekt zwischenzeitlich geändert hat.

8.11 Absturzwiederherstellung

Falls XIMOD Architect jemals unerwartet geschlossen wird, wird jedes Projekt mit ungespeicherten Änderungen in einen Wiederherstellungsordner unter dem Konfigurationsverzeichnis der Anwendung geschrieben (crash_recovery/<Datum>/<Mod-Name>/fomod/). Beim nächsten Start zeigt die Statusleiste an, wo die Sicherung gespeichert wurde; öffnen Sie diesen Ordner mit Datei → Ordner öffnen…, um Ihre Arbeit zurückzubekommen.

8.12 Eine übersichtlichere Benutzeroberfläche

Mehrere alltägliche Ärgernisse wurden behoben. Das Speichern eines Projekts, das noch keinen Ordner hat, fragt nun, wo es gespeichert werden soll, anstatt nichts zu tun, und Datei → Speichern unter… (Ctrl+Shift+S) verschiebt ein Projekt in einen anderen Ordner. Der Fenstertitel zeigt den Mod-Namen und einen Punkt ●, solange es ungespeicherte Änderungen gibt. Erfolge, Warnungen und Fehler erscheinen als Benachrichtigungen in der oberen rechten Ecke (Fehler bleiben, bis sie geschlossen werden), während die Statusleiste die letzte Meldung und die Anzahl der Schritte und Optionen behält (der Stammordner wird im Abschnitt „Arbeitsbereich“ des Inspektors angezeigt und der Mod-Name im Fenstertitel und in der Dokumentregisterkarte). Die Wahl einer Datei außerhalb des Stammordners wird nun gemeldet, anstatt stillschweigend ignoriert zu werden.
Terminologie — Die Benutzeroberfläche sagt nun durchgängig „Option“ für das, was das FOMOD-Format ein Plugin nennt (die in einer Gruppe angebotenen Auswahlmöglichkeiten), in allen Sprachen; „Plugin“ wird für Spiel-Plugin-Dateien (.esp, .esm, .esl) beibehalten. Die Design-Einstellung „System“ folgt dem Betriebssystem live: Das Umschalten des Betriebssystems zwischen hellem und dunklem Modus schaltet XIMOD sofort um, ohne Neustart, und die gewählte Schriftgröße bleibt in beiden erhalten. Schließlich werden die Datensätze (Spiele, Länder, Sprachen) gelesen, während der Startbildschirm angezeigt wird, anstatt danach, sodass das Hauptfenster früher erscheint.

8.13 Ein vorhandenes FOMOD übersetzen

Projekt → FOMOD übersetzen… öffnet ein eigenes Fenster, um die dem Benutzer angezeigten Zeichenketten eines Installationsprogramms in eine andere Sprache zu übersetzen: den Mod-Namen und die Beschreibung, die Namen von Schritten, Gruppen und Optionen sowie die Optionsbeschreibungen. Flags, Dateipfade und Typen werden niemals angetastet, da Mod-Manager sie wörtlich vergleichen. Laden Sie das aktive Projekt (es muss gespeichert sein) oder einen beliebigen Mod-Ordner mit „Mod-Ordner öffnen…“; wählen Sie die Ausgangs- und Zielsprachen und füllen Sie dann die Spalte „Übersetzung“ aus, entweder direkt in der Tabelle für Namen oder im unteren Bedienfeld für Beschreibungen (die Quelle wird nebeneinander angezeigt, wobei Leerzeichen und Zeilenumbrüche optional sichtbar gemacht werden). Filter (nicht übersetzt, zu überprüfen, mit Problemen, gesperrt), ein Suchfeld, ein Fortschrittsbalken und Ctrl+Enter (nächstes nicht übersetztes) helfen, den Überblick zu behalten. Autor und Website sind standardmäßig gesperrt: Gesperrte Zeichenketten werden unverändert geschrieben.

Probleme werden in jeder Zeile gekennzeichnet: geschützte Token (Dateinamen, URLs, wörtliche \n-Sequenzen), die in der Übersetzung fehlen, ein Zeilenumbruch in einem Namen, Zeichen, die XML nicht speichern kann, eine mit der Quelle identische Übersetzung oder derselbe Quelltext, der anderswo unterschiedlich übersetzt wurde. Blockierende Probleme verhindern den Export. Die Arbeit wird (Ctrl+S) als Begleitdatei gespeichert, fomod/translations/<mod>.<sprache>.ximod-translation, die das Fenster beim nächsten Öffnen des Ordners automatisch neu lädt; „Aus Ordner aktualisieren“ liest das FOMOD erneut und meldet neue, geänderte, verschobene und entfernte Zeichenketten, wobei die vorhandenen Übersetzungen beibehalten werden (geänderte Quellen werden als „zu überprüfen“ markiert).

„Die übersetzten Dateien exportieren“ schreibt info.xml und ModuleConfig.xml entweder in einen benachbarten Ordner fomod_<sprache> (Standard, die Originale bleiben unberührt) oder über die Originaldateien nach einer mit Zeitstempel versehenen .bak-Kopie. Der Export ist verlustfrei: Nur die übersetzten Zeichenketten ändern sich; Kommentare, Kodierung (UTF-8 oder UTF-16), unbekannte Elemente und Formatierung des ursprünglichen XML bleiben Byte für Byte erhalten. „Ursprüngliche Reihenfolge beibehalten“ erzwingt order="Explicit", damit nach Namen sortierte Listen nicht nach den übersetzten Namen neu sortiert werden. „Übersetztes in der Vorschau“ öffnet die Installationssimulation auf der übersetzten Kopie. Dieselben Vorgänge gibt es auf der Befehlszeile: ximod-architect translate extract | status | update | apply (siehe § 5).

Übersetzungspaket — Der dritte Exportmodus erstellt ein Archiv, das zum Hochladen auf Nexus bereit ist: standardmäßig ein „Nur-Patch“-Paket, das nur die übersetzten info.xml und ModuleConfig.xml (an ihrem ursprünglichen Pfad und in ihrer ursprünglichen Kodierung) sowie eine README in der Zielsprache enthält, die Benutzer über den Original-Mod installieren; die Option „Vollständiger Mod“ schließt stattdessen jede Datei des Mods ein (prüfen Sie, ob der Autor die Weiterverbreitung erlaubt). Das Format (.zip oder .7z) und der Dateiname folgen der Namensvorlage, mit den Token {name}, {version}, {LANG}, {lang}, {lang3} und {langname} (Standard „{name}_{LANG}“, z. B. MyMod_FR.zip). Das Archiv wird im Hintergrund mit dem üblichen Fortschrittsfenster erstellt.

Produktivität — Ein Übersetzungsspeicher pro Sprachpaar, der im Konfigurationsordner gespeichert wird, lernt jede Übersetzung, die Sie speichern; „Aus Speicher füllen“ füllt identische Zeichenketten im nächsten Mod vorab aus (als „zu überprüfen“ markiert), und das untere Bedienfeld schlägt die gespeicherte Übersetzung der ausgewählten Zeichenkette vor. „Auf identische übertragen“ kopiert die aktuelle Übersetzung auf jede andere Zeichenkette mit demselben Quelltext. Das Glossar (umschaltbar in der Werkzeugzeile) listet vorgeschriebene Begriffe auf — einen Begriff und seine erwartete Übersetzung, optional unter Beachtung der Groß-/Kleinschreibung, oder einen Begriff, der unübersetzt bleiben soll — und jede Zeile, die eine Regel verletzt, wird als Problem gekennzeichnet. „CSV exportieren“ / „CSV importieren“ tauschen die gesamte Tabelle (Schlüssel, Kontext, Quelle, Übersetzung, Status, Notiz) mit einer Tabellenkalkulation oder einem anderen Übersetzer aus; der Import ordnet Zeilen nach Schlüssel zu und toleriert Excels Trennzeichen.

Eindeutige Texte — Ein Umschalter in der Filterzeile listet eine Zeile pro eindeutigem Quelltext auf, mit einem ×N-Abzeichen, das die Größe der Gruppe angibt; das Übersetzen dieser Zeile übersetzt jede Zeichenkette mit demselben Text auf einmal, automatisch und ohne Nachfrage (gesperrte Zeichenketten ausgenommen), und eine Meldung sagt, wie viele aktualisiert wurden. Es ist der

schnellste Weg durch FOMODs, die dieselben Optionsnamen oder -beschreibungen auf vielen Seiten wiederholen. Ausgeschaltet zeigt die Tabelle jede Zeichenkette an, und „Auf identische übertragen“ bleibt für ein einmaliges Kopieren nur auf die nicht übersetzten verfügbar.
Tastatur und Leerraum — In der Tabelle bewegen ↑ und ↓ die ausgewählte Zeile, Page Up und Page Down springen um einen Bildschirm voller Zeilen, Home und End gehen zur ersten und letzten Zeile (solange in kein Übersetzungsfeld getippt wird; dieselben Tasten funktionieren im Fenster „Projekt-Texte“). „Leerraum und Zeilenumbrüche anzeigen“ macht jeden Leerraum im Quelltext sichtbar, sowohl in der Tabelle als auch im Detailbereich: · für ein Leerzeichen, ⍽ für ein geschütztes Leerzeichen, → für einen Tabulator, ␍ für einen Wagenrücklauf und ¶ an jedem Zeilenende.
Auf der Befehlszeile: ximod-architect translate package <root> <sidecar> [-o OUT] [--format zip|7z] [--full], translate export-csv / import-csv, translate tm-learn / tm-apply; und ein Batch-Manifesteintrag kann eine Liste „translations“ tragen (sidecar, mode, out, format, full), um die übersetzten Pakete zusammen mit dem Hauptarchiv zu erzeugen.

8.14 Ein Mod-Archiv öffnen

Datei → Archiv öffnen… (oder das Ablegen einer .zip-/.7z-Datei auf dem Fenster) entpackt das Archiv in den Arbeitsordner des Konfigurationsverzeichnisses (work/<Archivname>) im Hintergrund, mit dem üblichen Fortschrittsfenster, und öffnet dann das darin gefundene FOMOD — egal ob der fomod-Ordner oben im Archiv liegt oder unter einem einzelnen umschließenden Ordner. Ein bereits entpacktes Archiv wird unverändert wiederverwendet, sodass die dort vorgenommenen Bearbeitungen erhalten bleiben. .rar-Archive werden nicht unterstützt: Entpacken Sie sie zuerst mit 7-Zip. Das Übersetzungsfenster hat dieselbe Schaltfläche, „Mod-Archiv öffnen…“, die nur den fomod-Ordner entpackt (nach translate_work/<Archivname>); die Paketoption „Vollständiger Mod“ ist dann nicht verfügbar, da die Mod-Dateien dort nicht vorhanden sind.
Auf der Befehlszeile: ximod-architect extract <archive> [-o DIR] [--fomod-only] entpackt ein Archiv (standardmäßig in einen nach ihm benannten Ordner) und gibt den FOMOD-Stamm aus.

8.15 Importtreue

XIMOD liest und schreibt nun Konstrukte zurück, die es früher stillschweigend verwarf: die modweiten Voraussetzungen (moduleDependencies, bearbeitet auf der Registerkarte „Mod-Info“ unter „Mod-Voraussetzungen“ mit demselben Abhängigkeitseditor wie anderswo), die Attribute alwaysInstall und installIfUsable von Dateien und Ordnern (die Spalten „Immer“ und „Wenn verwendbar“ der Dateitabellen), die Titelposition und -farbe und die Kopfbildoptionen showImage, showFade und height (Mod-Info → „Erweiterter Header“) sowie die Reihenfolgeattribute Ascending/Descending von Schritten, Gruppen und Optionen, die wie verfasst beibehalten werden.
Was weiterhin nicht modelliert werden kann — Elemente außerhalb des Schemas, die XIMOD nicht kennt — wird beim Laden des Projekts gemeldet: Eine Warnung „Dieses FOMOD enthält N Konstrukte, die XIMOD nicht bearbeiten kann; sie werden beim Speichern des Projekts verworfen“ öffnet das Bedienfeld „Probleme“ mit einer Zeile pro Konstrukt und seinem Ort (Schritt, Option, bedingter Satz); ein Klick auf eine Zeile wählt den Knoten aus. Die Warnungen bleiben über Validierungen hinweg im Bedienfeld „Probleme“, solange das Projekt geöffnet ist. ximod-architect validate gibt dieselben Warnungen aus.

8.16 Sicherungen und Wiederherstellung

Jedes Mal, wenn ein Projekt über eine vorhandene Version gespeichert wird, werden die vorherigen info.xml und ModuleConfig.xml zuerst in fomod/backups/<Datum-Uhrzeit>/ kopiert (es wird nichts kopiert, wenn sie mit dem, was gleich geschrieben wird, identisch sind), und nur die N neuesten Ordner werden aufbewahrt — wobei N die Einstellung „Beizubehaltende Sicherungen“ ist. Dieser Ordner wird niemals in Distributionsarchive oder BA2-Pakete aufgenommen, und seine Dateien werden niemals als verwaist gemeldet. Beide XML-Dateien werden atomar geschrieben (temporäre Datei, flush, rename), sodass ein Absturz während eines Speichervorgangs kein halb geschriebenes Installationsprogramm hinterlassen kann.

Datei → Sicherung wiederherstellen… listet die Sicherungen auf, die neueste zuerst, jede mit der Anzahl der Unterschiede zum Projekt in seinem aktuellen Zustand; „Vergleichen“ öffnet das übliche Vergleichsfenster gegen diese Sicherung, „Wiederherstellen“ lädt sie als eine einzige rückgängig machbare Änderung in den Editor (es wird nichts auf die Festplatte geschrieben, bis Sie speichern), und „Alle Sicherungen löschen“ — durch einen zweiten Klick bestätigt — leert den Ordner. Zusätzlich zum Absturzschutz aus § 8.11 wird nun in regelmäßigen Abständen eine Wiederherstellungskopie jedes geänderten Projekts geschrieben („Wiederherstellungskopie automatisch alle N Minuten“); sie wird bei einem normalen Beenden gelöscht, sodass die Wiederherstellungsaufforderung weiterhin nur nach einem anormalen erscheint.
Auf der Befehlszeile: ximod-architect backups <root> --list, --restore <timestamp> oder --prune N; build und package erstellen ebenfalls eine Sicherung vor dem Schreiben.

8.17 Master als automatische Bedingungen

Wenn eine Plugin-Datei (.esp, .esm oder .esl) zu einer Option hinzugefügt wird — über die Dateitabelle oder per Drag-and-Drop —, liest XIMOD ihren Header: Wenn das Projekt noch keinen Autor hat, wird der Autor des Plugins eingetragen; und jeder Master, den das Plugin benötigt und der weder eine der eigenen Dateien des Spiels (pro Spiel in Categories.json unter „masters“ aufgeführt) noch von diesem Mod installiert ist, wird zu einer Dateibedingung „Active“ der Option, die dem ersten Abhängigkeitsmuster des Standardtyps der Option hinzugefügt wird (bei Bedarf erstellt). Ein Master, der bereits von einer Dateibedingung der Option benannt wird, wird niemals zweimal hinzugefügt. Die Einstellung „Die Master eines Plugins als Bedingungen hinzufügen“ schaltet dies aus.
Die Validierung liest den Header jedes vom Projekt installierten Plugins (erforderliche Dateien, Optionen, bedingte Sätze) und warnt vor jedem Master, der weder im Mod, noch eine Basisdatei des Spiels, noch als Abhängigkeit deklariert ist: „X benötigt Y, das weder in diesem Mod vorhanden noch als Abhängigkeit deklariert ist“. Die Warnung zielt auf den Knoten, sodass ein Klick darauf im Bedienfeld „Probleme“ die Option auswählt.

8.18 Light-Plugins (ESL) und der Plugin-Bericht

Die Validierung prüft auch die Light-Plugin-Regeln: eine Datei mit der Endung .esl, deren Light-Flag nicht gesetzt ist, ein als light gekennzeichnetes Plugin, dessen neue Datensätze das Limit überschreiten oder außerhalb des zulässigen FormID-Bereichs liegen, und — als Information — eine .esp, die als light gekennzeichnet werden könnte (es zählt die für das Plugin neuen Datensätze, das heißt diejenigen, deren FormID-Index gleich der Anzahl der Master ist, und vergleicht mit dem Limit des Spiels: standardmäßig 2048, 4096 für Starfield, pro Spiel mit „eslLimit“ in Categories.json anpassbar). Dateien über 64 MB werden nicht gezählt. Das CLI inspect <plugin> gibt dieselben Zahlen aus.
Werkzeuge → Plugin-Bericht… fasst alles in einer Tabelle zusammen: Datei, Art, Light-Flag, Master (fehlende in der Warnfarbe), neue Datensätze im Vergleich zum Limit und Eignung; nicht lesbare Dateien werden als solche angezeigt.

8.19 Reichhaltigerer Simulator und Szenarien

Die Zusammenfassungsseite der Installationsprogramm-Vorschau (Projekt → Vorschau) endet nun mit einem endgültigen Dateibaum: jede Datei, die im Data-Ordner des Spiels landen würde, Ordner von der Festplatte mit ihren Größen aufgeklappt, und die gesamte Installationsgröße. Wenn zwei Einträge auf demselben endgültigen Pfad landen, gewinnt die höhere Priorität (bei Gleichstand gewinnt die spätere Deklaration) und der Verlierer wird durchgestrichen angezeigt, mit der gewinnenden Option beim Überfahren mit der Maus. Bäume über 20 000 Einträge werden gekürzt und sagen das.
Der Dateibaum folgt denselben Regeln wie jeder Baum in XIMOD. Ein Rechtsklick auf einen Ordner öffnet die Untermenüs „Aufklappen“ und „Zuklappen“ (Alle / Ausgewählte / ab Ausgewähltem, wobei das letzte auf den Ordner und alles darunter wirkt, einschließlich des Inhalts installierter Archive). Solange kein Textfeld den Fokus hat, bewegen ↑ und ↓ einen Cursorring von Zeile zu Zeile, Home und End springen zu den Enden, → öffnet einen Ordner (oder tritt in einen geöffneten ein), ← schließt ihn (oder klettert von einer Datei zu ihm zurück), und Enter wählt im Hauptfenster die Komponente aus, die

die Zeile installiert: die Option, die erforderlichen Dateien oder den bedingten Satz — bei einem Ordner die Komponente seiner ersten Datei.
Die Vorschau öffnet außerdem, wie es ein Mod-Manager tut, auf einer Mod-Informationsseite vor dem ersten Schritt: das Kopfbild (unter Beachtung der Optionen showImage und height), der Titel in seiner Farbe, der Autor, die Version, der Website-Link und die Beschreibung aus info.xml. „Weiter“ führt zum ersten sichtbaren Schritt, „Zurück“ kehrt dorthin zurück, und der Fortschrittsbalken zählt sie als erste Seite.
Eine Szenario-Zeile ermöglicht es, die aktuellen Auswahlen und Dateiannahmen unter einem Namen zu speichern (in fomod/scenarios/<Name>.json gespeichert, nach Optionsnamen, damit sie ein Neuanordnen überstehen), sie neu zu laden oder zu löschen; ein Szenario, dessen Auswahlen keiner Option mehr entsprechen, meldet, wie viele nicht gefunden wurden. Der Szenario-Ordner wird niemals in Distributionsarchive aufgenommen.
Die Validierung meldet nun außerdem, als Information, die Schritte, Optionen und bedingten Sätze, die niemals erreicht werden können, weil sie einen Flag-Wert prüfen, den keine Option setzt (Schritte: keine Option in einem früheren Schritt).
Auf der Befehlszeile: ximod-architect simulate <root> [--scenario <name|file.json>] [--json] gibt die sichtbaren Schritte, die Auswahlen, die endgültige Installationsliste mit Größen und Überschreibungen und die Gesamtsumme aus; ein Batch-Manifesteintrag kann „scenarios“: [...] tragen, um sie erneut abzuspielen (ein Eintrag schlägt fehl, wenn ein Szenario nicht mehr passt).

8.20 Installationsgröße pro Option

Nach einer Validierung (die die Dateien im Hintergrund misst) zeigt jede Option ihre Installationsgröße und Dateianzahl über ihrer Dateitabelle an — fehlende Quellen werden in der Warnfarbe gezählt — und der Projektbaum trägt die Größe nach dem Namen der Option. Größen werden nach Dateiänderungszeit zwischengespeichert und bei jeder Validierung aktualisiert. ximod-architect validate <root> --sizes gibt die Gesamtsummen für die Optionen, die erforderlichen Dateien und die bedingten Sätze aus.

8.21 Nexus-Beschreibungsgenerator

Werkzeuge → Nexus-Beschreibung… erstellt den Text der Nexus-Seite des Mods aus dem Projekt: Titel und Beschreibung, Voraussetzungen (die Master der installierten Plugins, die das Spiel und der Mod nicht bereitstellen, plus die deklarierten Dateiabhängigkeiten), Installationsoptionen (ein Aufzählungspunkt pro Schritt, Gruppen mit ihrem Auswahltyp, Optionen mit dem ersten Satz ihrer Beschreibung), einen Installationshinweis und ein Änderungsprotokoll, berechnet gegenüber einer früheren Version des FOMOD, die mit „Frühere Version…“ gewählt wird (Schritte, Optionen und Dateien, die hinzugefügt oder entfernt wurden, Metadatenänderungen). Wählen Sie BBCode (Nexus) oder Markdown, kreuzen Sie die gewünschten Abschnitte an, wählen Sie die Sprache unter den im Projekt gefundenen Übersetzungs-Begleitdateien und dann „Kopieren“ oder „Speichern unter…“. Auf der Befehlszeile: ximod-architect nexus-desc <root> [--format bbcode|md] [--previous <root>] [--lang <code>] [-o FILE].

8.22 Ein bearbeitbarer Bedingungseditor

Der Bedingungseditor aus § 8.7 wirkt nun auf das Projekt. Jede Zeile „gesetzt von“ und „verwendet von“ ist ein Link, der den Knoten im Hauptfenster auswählt. Jedes Flag hat „Umbenennen…“ (ein Inline-Feld; der neue Name wird auf jede Option, die das Flag setzt, und jede Schrittsichtbarkeit, jedes Optionsmuster, jeden bedingten Satz und jede Mod-Voraussetzung, die es prüft, angewendet, als eine einzige rückgängig machbare Änderung; ein Name, der bereits von einem anderen Flag verwendet wird, wird abgelehnt) und „Alle Verwendungen löschen“. Unter jedem Flag werden die Werte, auf die es gesetzt wird, und die Werte, mit denen es geprüft wird, aufgelistet: ein Wert, der geprüft, aber nie gesetzt wird, wird in der Warnfarbe angezeigt, ein Wert, der gesetzt, aber nie geprüft wird, in Grau. Die Validierung meldet dieselben Befunde — eine Warnung für einen geprüften Wert, den keine Option

setzt (zielt auf den Knoten, der ihn prüft), und eine Information für ein Flag, das gesetzt, aber nie geprüft wird.
Der „Bedingungsersteller“ am unteren Rand des Fensters bearbeitet die Bedingung des im Hauptfenster ausgewählten Knotens — die Sichtbarkeit eines Schritts, das Abhängigkeitsmuster einer Option (Muster wählen, eines erstellen, seinen Typ ändern), einen bedingten Dateisatz oder die Mod-Voraussetzungen. Er zeigt die Bedingung als Satz an (WENN Flag A = B UND Datei C Active ist DANN …) über einer Zeile pro Bedingung: Art (Flag oder Datei), Name und Wert mit aus dem Projekt übernommenen Vorschlägen (bekannte Flags und ihre Werte, Dateinamen, Active/Inactive/Missing), eine Entfernen-Schaltfläche und „Bedingung hinzufügen“. Änderungen werden live angewendet und sind rückgängig machbar.
Der Editor ist nun ein unabhängiges Fenster: Er kann neben das Hauptfenster oder auf einen zweiten Bildschirm verschoben werden, behält seine Position und Größe von einer Sitzung zur nächsten und schließt sich mit seiner Titelleistenschaltfläche. Die Flag-Liste funktioniert wie der Projektbaum. Ein Rechtsklick auf eine Flag-Überschrift öffnet die Untermenüs „Aufklappen“ (Alle / Ausgewählte) und „Zuklappen“ (Alle / Ausgewählte). Solange kein Textfeld den Fokus hat, bewegen ↑ und ↓ einen Cursorring von einer Zeile zur nächsten — Flag-Überschriften und ihre Zeilen „gesetzt von“ / „verwendet von“ —, → klappt ein geschlossenes Flag auf (oder tritt in ein geöffnetes ein), ← klappt ein geöffnetes Flag zu (oder klettert von einer seiner Zeilen zu ihm zurück), und Enter zeigt die Zeile im Inspektor an: den Knoten einer Zeile oder die erste Stelle eines Flags. Im Projektbaum tut Enter dasselbe: Es wählt die Zeile unter dem Cursor aus, und der Inspektor folgt.

8.23 Projekt-Texttabelle

Werkzeuge → Projekt-Texte… listet jeden Namen und jede Beschreibung des Projekts (Mod-Informationen, Schritte, Gruppen, Optionen) in einer Tabelle auf: Ort, Feld und Text, wobei der Text direkt bearbeitet werden kann (eine Zeile für Namen, mehrere Zeilen für Beschreibungen). Ein Suchfeld, ein Art-Filter (alle, Namen, Beschreibungen) und ein Umschalter „Nur Duplikate“ grenzen die Zeilen ein; identische Texte (unter Ignorieren von Groß-/Kleinschreibung und Abstand) werden mit ×N gekennzeichnet, mit den anderen Orten beim Überfahren mit der Maus, was hilft, kopierte Beschreibungen zu erkennen. Die Suchen/Ersetzen-Zeile (Groß-/Kleinschreibung beachten, ganzes Wort) ersetzt in der aktuellen Zeile oder in jeder sichtbaren Zeile. Ein Doppelklick auf einen Ort wählt den Knoten im Hauptfenster aus. Auf der Befehlszeile: ximod-architect strings <root> [--duplicates] [--csv FILE].

8.24 BSA- und BA2-Archive lesen

XIMOD liest nun die Inhaltsverzeichnisse von Bethesda-Archiven, ohne sie zu entpacken: BSA-Versionen 103 (Oblivion), 104 (Skyrim, Fallout 3/New Vegas) und 105 (Skyrim Special Edition) sowie allgemeine (GNRL) und Textur-(DX10-)BA2-Archive von Fallout 4 und Starfield (Versionen 1, 2, 3, 7 und 8). Morrowind-Archive werden mit einer klaren Meldung abgelehnt. Es werden nur die Eintragsnamen und -größen gelesen; komprimierte Einträge werden mit ihrer gepackten Größe aufgeführt.
Dies speist drei Stellen. Die Erkennung von Zielkonflikten (§ 8.1) schaut nun in die Archive, die eine Option installiert: zwei Optionen, deren Archive dasselbe Asset enthalten, werden als Warnung gemeldet („die Archiv-Ladereihenfolge des Spiels entscheidet“), und ein Asset, das sowohl gepackt als auch lose installiert ist, als Information („die lose Datei gewinnt“). Der endgültige Dateibaum des Simulators (§ 8.19) listet den Inhalt jedes installierten Archivs darunter auf, markiert mit „(im Archiv)“, und die Gesamtsumme zeigt, wie viel in Archiven gepackt ist (ohne es doppelt zu zählen). Und Werkzeuge → Archivinhalt… (oder die Augen-Schaltfläche neben einem Archiv in einer Dateitabelle) öffnet eine Tabelle der Einträge — Format, Anzahl, gesamte entpackte Größe, Pfad, Größe, komprimiert — mit einem Suchfeld. Auf der Befehlszeile: ximod-architect archive <file.bsa|file.ba2> [--json]; validate gibt nun auch die sicheren Zielkonflikte aus.
Der experimentelle BA2-Writer (CLI ba2) wird nun validiert, indem seine Ausgabe mit dem neuen Reader zurückgelesen wird: Pfade, Größen und Namens-/Ordner-Hashes stimmen überein.

8.25 Verschachtelte Bedingungsgruppen und Versionsbedingungen

Bedingungen sind nicht mehr auf eine flache Liste mit einem einzigen Operator beschränkt. Jede Stelle, die eine Bedingung enthält — die Sichtbarkeit eines Schritts, die Abhängigkeitsmuster einer Option, ein bedingter Dateisatz und die Mod-Voraussetzungen —, ist nun eine Gruppe, die Bedingungen und verschachtelte Gruppen enthalten kann, jede mit ihrem eigenen And-/Or-Operator, in beliebiger Tiefe, genau wie das FOMOD-Schema es erlaubt: „(DataVersion = Standard OR DataVersion = Undelayed) AND DataPreset = Aurelia“ bleibt wie geschrieben erhalten, wird geladen, bearbeitet und unverändert gespeichert (verifiziert an Aurelia UI und seinen 234 bedingten Sätzen). Zwei Bedingungsarten wurden ebenfalls hinzugefügt: Spielversion (gameDependency) und Mod-Manager-Version (fommDependency), jede mit einer Mindestversion.
In den Abhängigkeitseditoren erstellt „Gruppe hinzufügen“ eine verschachtelte Gruppe als eingerückten Rahmen mit ihrem eigenen Operator, ihren eigenen Bedingungen, Schaltflächen „Bedingung hinzufügen“ / „Gruppe hinzufügen“ / „Gruppe entfernen“; der Bedingungsersteller (§ 8.22) zeigt verschachtelte Gruppen in Klammern in seinem Satz an und bearbeitet sie mit denselben Steuerelementen. Der Simulator, die Validierung (geprüfte, aber nie gesetzte Flags, unerreichbare Schritte), die Konflikterkennung und die Nexus-Beschreibung verstehen alle verschachtelte Gruppen. Ältere Projektvorlagen, die mit dem flachen Layout gespeichert wurden, lassen sich weiterhin laden.
Zwei kleinere Änderungen in derselben Version: Ein Menü, das durch Gleiten des Zeigers entlang der Menüleiste geöffnet wird, schließt sich nun, wenn der Zeiger es verlässt (ein durch Klick geöffnetes Menü bleibt, bis etwas angeklickt wird); und die Schriftarten, die zur Anzeige der Schriften des Projekts selbst — Japanisch, Koreanisch, Chinesisch, Arabisch, Thai… in Namen und Beschreibungen — und der gerade bearbeiteten Übersetzung benötigt werden, werden automatisch geladen, unabhängig von der Sprache der Benutzeroberfläche, sodass keine Quadrate mehr anstelle dieser Zeichen erscheinen, auch in den Sprachlisten der Übersetzungs- und Nexus-Fenster.
Die Menüs sind in Datei, Projekt, Werkzeuge und Hilfe gegliedert, mit einer Symbolleiste für die alltäglichen Aktionen, und F1 öffnet dieses Handbuch. Gruppenauswahltypen, Optionstypen, Operatoren und Designs werden mit Bezeichnungen in verständlicher Sprache angezeigt („Genau eines (erforderlich)“, „Empfohlen“, „Alle Bedingungen (AND)“…) mit einer Erklärung beim Überfahren mit der Maus, und jeder Abschnitt des Optionsbedienfelds hat eine ⓘ-Markierung, die das Konzept erklärt. Bestätigungsdialoge dimmen das Fenster, akzeptieren Enter und Esc und scrollen, wenn die Liste der Probleme lang ist.

8.26 Spiele und ihre Kategorien bearbeiten

Werkzeuge → Spiele / Kategorien… öffnet ein eigenständiges Fenster, um die Liste der im Reiter Mod Info angebotenen Spiele zu verwalten, ohne Categories.json von Hand zu bearbeiten. Im linken Bereich werden die Spiele aufgelistet; mit Spiel hinzufügen, Spiel löschen sowie den Schaltflächen Nach oben / Nach unten ändern Sie die Liste und ihre Reihenfolge. Für das ausgewählte Spiel zeigt der Editor dessen Kennung (einen eindeutigen internen Schlüssel), seinen Namen, seinen Nexus-slug (für die Links und die Nexus-Beschreibung), die Datensatzgrenze für Light-Plugins, die Basis-Masterdateien (eine pro Zeile — die Master des Spiels selbst und die offiziellen DLCs) und die Nexus-Kategorien (eine pro Zeile, in der Anzeigereihenfolge). Die Eingaben werden während der Eingabe geprüft: Ein Spiel ohne Kennung, eine doppelte Kennung oder ein fehlender Name wird markiert, und Speichern bleibt deaktiviert, bis jedes Problem behoben ist. Speichern schreibt die Datei — in die mitgelieferte assets/data/Categories.json, sofern sie beschreibbar ist, andernfalls in eine benutzereigene Kopie, die die Anwendung zuerst liest — und die neue Liste wird sofort wirksam; Schließen verwirft nach einer Bestätigung die nicht gespeicherten Änderungen.

8.27 Baumfarben

Werkzeuge → Baumfarben… gibt jeder Zeilenart des Projektbaums eine eigene Farbe — Mod-Informationen, Schritte, Gruppen, Optionen, erforderliche Dateien und bedingte Installationen —, sodass sich die Struktur eines großen Installationsprogramms auf einen Blick erfassen lässt, was auch hilft, wenn die Augen nicht mehr so gut sind wie früher (kombinieren Sie dies mit der Schriftgröße der Benutzeroberfläche, § 4.5). Das Fenster listet die sechs Arten mit einem Farbfeld, dem Symbol in dieser Farbe und dem RGB-Wert auf; Farben zurücksetzen stellt die Standardwerte wieder her. Es gibt einen Farbsatz pro Design: Das Fenster bearbeitet den Satz des gerade verwendeten Designs und weist darauf hin, und ein Wechsel zwischen dunklem und hellem Design (§ 4.5) wechselt auch den Satz. Die Farben werden in Config.ini gespeichert (Anhang A). Die ausgewählte Zeile behält immer die Standardtextfarbe, damit sie auf dem Auswahlhintergrund lesbar bleibt.
Ein Klick auf ein Farbfeld öffnet das Fenster Farbe auswählen, denselben Farbwähler wie in XIMOD Translator. Seine Registerkarte Indexfarbe zeigt den AutoCAD-Farbindex: das 24 × 10-Raster aus Farbtönen und Schattierungen, die neun Standardfarben, die Graustufenreihe und ein Feld Farbe, in das ein Index von 1 bis 255 eingegeben werden kann. Seine Registerkarte True Color bietet eine Fläche für Farbton / Sättigung, einen Schieberegler für die Leuchtdichte, die Felder Farbton, Sättigung und Leuchtdichte (H, S, L), die Rot-, Grün- und Blauwerte der aktuellen Farbe sowie ein Feld RGB-Farbe, in das ein Wert wie 255,128,0 eingegeben werden kann. OK übernimmt die Farbe sofort — der Baum wird hinter dem Fenster neu gezeichnet — und speichert sie; Abbrechen oder Esc behält die vorherige bei.

8.28 Dateidialoge merken sich, wo Sie waren

Wenn Sie einer Option Quellen hinzufügen (Datei hinzufügen, Ordner hinzufügen) oder nach einem Bild suchen, beginnt der Dialog nicht mehr jedes Mal im Stammverzeichnis des Projekts: Er öffnet sich im übergeordneten Ordner der zuletzt gewählten Datei bzw. des zuletzt gewählten Ordners. Wählen Sie Feature D\A für eine Option, und der nächste Dialog öffnet sich in Feature D, mit B und C direkt daneben; wählen Sie Dateien innerhalb eines Ordners, und der nächste Dialog öffnet sich in diesem Ordner. Diese Erinnerung gilt für die Dauer der Sitzung und wird zurückgesetzt, wenn das Projekt wechselt; liegt der gemerkte Ordner außerhalb des Stammverzeichnisses oder existiert er nicht mehr, greift der Dialog auf das Stammverzeichnis zurück. Bei einem großen Installationsprogramm bleibt der Explorer der schnellste Weg: Öffnen Sie den Funktionsordner einmal und legen Sie jeden Unterordner auf seiner Option ab (§ 8.5).

8.29 Preset-Assistent

Projekt → Preset-Assistent… baut im aktuellen Projekt das in § 7.5 erklärte Muster „Preset + Anpassen“ auf: Ein Benutzer kann ein Preset wählen und fertig sein, ein Preset wählen und trotzdem einige Funktionen anpassen oder alles von Hand zusammenstellen — ohne dass jemals eine Datei zweimal

installiert wird. Von Hand bedeutet das Flags auf jeder Option, dynamische Typen, Sichtbarkeitsbedingungen und eine bedingte Installation pro Option; der Assistent erledigt all das in einem einzigen Schritt, der sich rückgängig machen lässt.
Markieren Sie in der Registerkarte Aus vorhandenen Seiten die Schritte, die Funktionsseiten sind (aus ihren Optionen wählen die Presets aus), prüfen Sie die generierten Namen — Schritt „Installationsmodus“, Preset-Gruppe, Gruppe „Anpassen“, Beschriftung der Optionen „Anpassen“, Preset-Flag, Name der Option „Keine“ — und definieren Sie die Presets: Preset hinzufügen erstellt eine Spalte, und die Matrix zeigt jede Option jeder markierten Seite als Zeile. Wählen Sie in einer exklusiven Gruppe (SelectExactlyOne / SelectAtMostOne) pro Preset eine Option oder (keine); kreuzen Sie in einer Mehrfachauswahl-Gruppe beliebig viele an. Die Leiste unten meldet das erste Problem (keine Seite markiert, kein Preset, ein fehlender Name, zwei Presets mit demselben Namen), und Anwenden bleibt deaktiviert, bis es behoben ist.
Anwenden fügt zuerst den Schritt „Installationsmodus“ ein (die Presets, die jeweils die Flags ihrer Auswahl setzen, und pro Seite eine Option „Anpassen“, die diese Seite öffnet und ihre Flags zurücksetzt), macht jede Funktionsseite nur dann sichtbar, wenn ihre Option „Anpassen“ angekreuzt ist, fügt den Optionen die Flags hinzu und kennzeichnet die von einem Preset gewählten Varianten unter diesem Preset als Recommended, fügt exklusiven Gruppen eine Option „Keine“ hinzu (standardmäßig empfohlen, optional unter den Presets, die etwas auswählen) und verschiebt die Dateien der Optionen in die Registerkarte Bedingte Installationen, ein Satz pro Flag-Wert. Die Meldung fasst zusammen, was getan wurde; Bearbeiten → Rückgängig macht den gesamten Vorgang rückgängig. Prüfen Sie das Ergebnis anschließend mit Projekt → Installationsprogramm-Vorschau… (Preset allein, Preset plus eine angepasste Seite, Seite allein) und Projekt → FOMOD validieren, das dieses Muster kennt (§ 8.1).
Die Registerkarte Neues Gerüst ist für ein Projekt gedacht, das noch keine Seiten hat: Beschreiben Sie jede Funktion durch einen Seitennamen, die Art der Auswahl (exklusiv oder mehrfach) und eine Variante pro Zeile; Seiten erstellen fügt sie dann dem Projekt hinzu — ohne Dateien — und wechselt zur ersten Registerkarte, in der diese Seiten bereits markiert sind. Definieren Sie dort die Presets, wenden Sie sie an und weisen Sie die Dateien danach zu: auf den Seiten für das, was von Hand gewählt wird, und in den vom Assistenten erstellten bedingten Installationen für das, was die Presets installieren.

Anhänge

Anhang A — Die Datei Config.ini

Die Konfiguration wird in der Datei Config.ini gespeichert, die sich neben der ausführbaren Datei befindet. Hier ist ein vollständiges Beispiel:

```
; XIMOD Architect Configuration
[General]
Locale=fra
Country=FRA
FirstStart=1
Theme=Dark
FontSize=14
ReplaceNewlines=1
MaxRecentFiles=10
SplashScreenSeconds=2

[Window]
WindowWidth=1280
WindowHeight=800

[WindowPositions]
WinPos_ximod_translation=120,80
WinSize_ximod_translation=1100,700
WinPos_ximod_preview=200,140
WinSize_ximod_preview=900,650

[Scripts]
PreSaveScript=
PostSaveScript=

[RecentFiles]
RecentFile0=C:\Mods\MyMod
```

Zwei Schlüssel werden zu den vorherigen hinzugefügt: Country (ISO 3166-1 alpha-3-Code des gewählten Landes) und FirstStart (siehe § 2.5).

Der Abschnitt [WindowPositions] merkt sich die Geometrie der freien Fenster (siehe § 2.6). Für jedes Fenster speichert WinPos_<id> seine Position (x,y) und WinSize_<id> seine Größe (Breite,Höhe). Diese Einträge werden erst erstellt, sobald der Benutzer das Fenster tatsächlich verschoben oder in der Größe verändert hat; ohne sie öffnet sich das Fenster zentriert auf dem Hauptfenster. Die Größe des Hauptfensters wird in [Window] (WindowWidth / WindowHeight) gespeichert, seine Position jedoch nicht.

Anhang B — Tastaturkürzel

Unter macOS entspricht die Ctrl-Taste Cmd.

Globale Tastenkürzel

Tastenkürzel	Aktion
Ctrl+N	Neues Projekt.
Ctrl+O	Einen Ordner öffnen.
Ctrl+Shift+O	Eine Datei öffnen.
Ctrl+S	Speichern (das FOMOD generieren).
Ctrl+,	Die Einstellungen öffnen.
Ctrl+Q	Die Anwendung beenden.
F1	Das Fenster „Über“ anzeigen.

Fenster und Dialoge

Tastenkürzel	Aktion

Tastenkürzel	Aktion
Esc	Schließt das aktive freie Fenster (Länderauswahl, Eigenschaften, XML-Editor) oder den modalen Dialog (Einstellungen, Über, Bestätigung, Prüfbericht, Skripte). Bei einer Bestätigung wirkt es als „Abbrechen“. Keine Wirkung auf den Übersetzungseditor oder das Hauptfenster.

Listennavigation — Eigenschaften und Übersetzungseditor

Taste	Aktion
↑ / ↓	Bewegt die Auswahl eine Zeile nach oben / unten.
Page Up / Page Down	Scrollt um einen Bildschirm; die Schrittweite passt sich an die Fensterhöhe an.
Home / End	Springt zur ersten / letzten Zeile (die letzte Zeile wird vollständig in den sichtbaren Bereich gebracht).
Enter	Eigenschaften: wählt das hervorgehobene Element aus. Übersetzungseditor: platziert den Cursor im Eingabefeld der Zeile.
Tab / Shift+Tab	Eigenschaften: nächste / vorherige Zeile (niemals auf der Flagge). Übersetzungseditor (während der Bearbeitung): nächstes / vorheriges Übersetzungsfeld.

Flaggenraster — Fenster „Länderauswahl“

Taste	Aktion
↑ ↓ ← → (oder 8 2 4 6)	Bewegt den Auswahlrahmen innerhalb des Rasters.
Page Up / Page Down	Scrollt um einen Bildschirm voller Flaggen.
Home / End	Springt zur ersten / letzten Flagge (die letzte Zeile wird vollständig in den sichtbaren Bereich gebracht).
Tab / Shift+Tab	Nächste / vorherige Flagge.
Enter	Bestätigt das hervorgehobene Land.
Esc	Schließt das Fenster.

Anhang C — Glossar

Begriff	Definition
FOMOD	Standardformat für Mod-Installationsprogramme für Bethesda-Spiele, basierend auf info.xml und ModuleConfig.xml.
Schritt	Seite des Installationsassistenten, die eine oder mehrere Gruppen enthält.
Gruppe	Satz von Optionen (Plugins), die einen Auswahltyp teilen.
Plugin	Dem Benutzer angebotene Option, die mit zu installierenden Dateien verknüpft ist.
Flag	Bedingungsflag (Name-Wert-Paar), das zur Steuerung der Installation verwendet wird.
Endonym	Name eines Landes oder einer Sprache, so wie er in dieser Sprache geschrieben wird.
ISO 639-3	Dreibuchstabiger Sprachcode, der die Übersetzungsordner benennt (z. B. fra, eng).

Begriff	Definition
ISO 3166-1 alpha-3	Dreibuchstabiger Ländercode (z. B. FRA, USA).
Stammverzeichnis	Arbeitsordner, der die Mod-Dateien und den generierten Unterordner fomod enthält.

Anhang D — Vollständiges FOMOD-Beispiel

Dieser Anhang stellt ein fiktives, aber vollständiges Projekt vor — „Dawnlight“, eine Beleuchtungs-Mod für Skyrim Special Edition —, das das gesamte Spektrum der Möglichkeiten von XIMOD Architect in die Praxis umsetzt: erforderliche Dateien, mehrere Schritte, eine Schrittsichtbarkeitsbedingung, die fünf Gruppentypen, die fünf Optionstypen, Bedingungsflags, dynamischer Optionstyp (Abhängigkeitsmuster) und bedingte Installationen. Die beiden untenstehenden Dateien bilden ein einziges Projekt und werden so wiedergegeben, wie XIMOD sie generiert (vereinfachte Einrückung).

D.1 info.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>Dawnlight</Name>
    <Author>Patrice</Author>
    <Version>1.2.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/98765</Website>
    <Description>Improved lighting, candles and weather for Skyrim SE.</Description>
    <Groups>Visuals and Graphics</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

D.2 ModuleConfig.xml

Vom Beispiel abgedeckte Funktionen:

- Erforderliche Dateien: requiredInstallFiles (eine Datei und ein Ordner).
- Schritt 1: SelectAll-Gruppe (Option Required) und SelectExactlyOne (Optionen Recommended / Optional, res-Flag).
- Schritt 2: SelectAny-Gruppe (candles- und moon-Flags) und SelectAtMostOne.
- Schritt 3: Sichtbarkeitsbedingung (<visible>) dann SelectAtLeastOne-Gruppe mit einer Option mit dynamischem Typ (dependencyType: Recommended wenn candles=On, NotUsable wenn eine Datei fehlt) und einer CouldBeUsable-Option.
- Bedingte Installationen: zwei Muster (And- und Or-Operatoren, Flag- und Datei-Abhängigkeiten, Zustände Active / Inactive / Missing).

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Dawnlight</moduleName>
  <moduleImage path="fomod\header.png"/>
  <requiredInstallFiles>
    <file source="Dawnlight.esp" destination="Dawnlight.esp" priority="0"/>
    <folder source="core\meshes" destination="meshes" priority="0"/>
  </requiredInstallFiles>
  <installSteps order="Explicit">
    <installStep name="Main installation">
      <optionalFileGroups>
        <group name="Core module" type="SelectAll">
          <plugins order="Explicit">
            <plugin name="Base files">
              <description>Scripts and meshes essential to the mod.</description>
              <files>
                <folder source="core\scripts" destination="scripts" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Required"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
        <group name="Texture resolution" type="SelectExactlyOne">
```

```
        <plugins order="Explicit">
          <plugin name="2K">
            <description>2K textures, good quality/performance balance.</description>
            <image path="fomod\2k.png"/>
            <conditionFlags>
              <flag name="res">2K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\2k" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Recommended"/>
            </typeDescriptor>
          </plugin>
          <plugin name="4K">
            <description>4K textures, for powerful setups.</description>
            <image path="fomod\4k.png"/>
            <conditionFlags>
              <flag name="res">4K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\4k" destination="textures" priority="1"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="Ambience options">
    <optionalFileGroups>
      <group name="Lighting effects" type="SelectAny">
        <plugins order="Explicit">
          <plugin name="Dynamic candles">
            <description>Adds candles with flickering light.</description>
            <conditionFlags>
              <flag name="candles">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\candles" destination="meshes" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Enhanced moon">
            <description>Increases the intensity of the moonlight.</description>
            <conditionFlags>
              <flag name="moon">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\moon" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
      <group name="Sound ambience" type="SelectAtMostOne">
        <plugins order="Explicit">
          <plugin name="Night sounds">
            <description>Adds a nighttime sound ambience.</description>
            <files>
              <folder source="opt\sound" destination="sound" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="ENB compatibility">
```

```
    <visible>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <fileDependency file="KnownConflict.esp" state="Inactive"/>
      </dependencies>
    </visible>
    <optionalFileGroups>
      <group name="ENB preset" type="SelectAtLeastOne">
        <plugins order="Explicit">
          <plugin name="Light ENB">
            <description>Low-cost ENB preset.</description>
            <files>
              <file source="enb\light.ini" destination="enblocal.ini" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Ultra ENB">
            <description>High-end ENB preset (dynamic type).</description>
            <files>
              <file source="enb\ultra.ini" destination="enblocal.ini" priority="1"/>
            </files>
            <typeDescriptor>
              <dependencyType>
                <defaultType name="Optional"/>
                <patterns>
                  <pattern>
                    <dependencies operator="And">
                      <flagDependency flag="candles" value="On"/>
                    </dependencies>
                    <type name="Recommended"/>
                  </pattern>
                  <pattern>
                    <dependencies operator="Or">
                      <fileDependency file="EnbHelper.dll" state="Missing"/>
                    </dependencies>
                    <type name="NotUsable"/>
                  </pattern>
                </patterns>
              </dependencyType>
            </typeDescriptor>
          </plugin>
          <plugin name="Experimental extension">
            <description>Additional effects, not guaranteed.</description>
            <files>
              <folder source="enb\extra" destination="textures" priority="2"/>
            </files>
            <typeDescriptor>
              <type name="CouldBeUsable"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
</installSteps>
<conditionalFileInstalls>
  <patterns>
    <pattern>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <flagDependency flag="candles" value="On"/>
      </dependencies>
      <files>
        <folder source="cond\candles4k" destination="textures" priority="0"/>
      </files>
    </pattern>
    <pattern>
      <dependencies operator="Or">
        <flagDependency flag="moon" value="On"/>
        <fileDependency file="SkyUI_SE.esp" state="Active"/>
      </dependencies>
      <files>
        <file source="cond\patch.esp" destination="patch.esp" priority="0"/>
      </files>
    </pattern>
```

```
    </patterns>
  </conditionalFileInstalls>
</config>
```

Anhang E — Lizenz und Danksagungen

Der Quellcode von XIMOD Architect wird unter der MIT-Lizenz verteilt (© 2025-2026). Die untenstehenden Drittanbieterkomponenten behalten ihre eigene Lizenz; die vollständigen Einzelheiten und Danksagungen erscheinen in der Datei THIRD-PARTY-NOTICES, die mit der Anwendung bereitgestellt wird.

Komponente	Quelle	Lizenz
XIMOD Architect-Code	—	MIT
Noto-Schriftarten	Google Fonts / Noto-Projekt	SIL Open Font License 1.1
Flaggen (SVG)	drapeauxdespays.fr	Lizenzfrei (kommerzielle Nutzung inbegriffen)
ISO 3166-1 alpha-3-Ländercodes	ISO — iso.org/obp	Freier Zugang
ISO 639-3-Sprachcodes	SIL — iso639-3.sil.org	Namensnennung erforderlich (siehe unten)
Rust-Bibliotheken	crates.io-Ökosystem	MIT / Apache-2.0 (größtenteils)

ISO 639-3-Namensnennung: Die Sprachcodes stammen von iso639-3.sil.org (Quelle, die anzugeben ist). Die von XIMOD hinzugefügten Daten — Endonyme, zugehörige Schriftart, Zuordnungen Land ↔ Sprache — sind nicht Teil des Standards ISO 639-3, und die Bezeichner des Standards werden nicht verändert. Die Anwendung darf zudem keinerlei Mittel zur Weiterverbreitung des vollständigen Satzes von Codes bereitstellen.